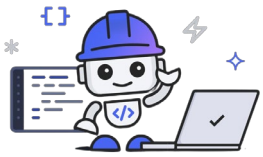
watsonx Challenge 2026
Bob Quantum Computing Agent (BQCA)

Quantum appliance solution powered by IBM Bob & Inspired by IBM watsonx Challenge

IBM Quantum Team Solution Built with IBM Bob

Whitepaper rev. v1.3.0

TEAM



+



Bob Quantum Team

©2026

Team Authors:

- *Akpojotor Shemi (IBM Quantum Qiskit Advocate)*
- *Nic Ponder (Aspiring Quantum Graduate Candidate)*
- *Taylor Sperry (IBM Quantum Ambassador Candidate)*
- *Jeffrey Chijioke-Uche, PhD*
  - *IBM Computer Scientist*
  - *Inventor, Quantum Computing Qiskit Connector™*
  - *IBM Quantum Ambassador*
  - *IBM Quantum Qiskit Advocate*
  - *IEEE Senior Member (Computing Intelligence)*
  - *IBM watsonx Challenge 2026 Team Lead*

16+	4	Quantum	v1.0.6
Active Bob Files Created as part of the BQCA solution	Custom Bob Engine Categories: modes, skills, rules, tools	Quantum Computing Use Cases Focus	Runtime Bob Baseline for Bob Shell

2026 IBMer watsonx Challenge Solution with IBM Bob

BQCA Solution User Interface Menu – Driven by IBM Bob Shell

Figure 1.0: Real BQCA Solution Main Menu (live usability proof) built with IBM Bob Shell:

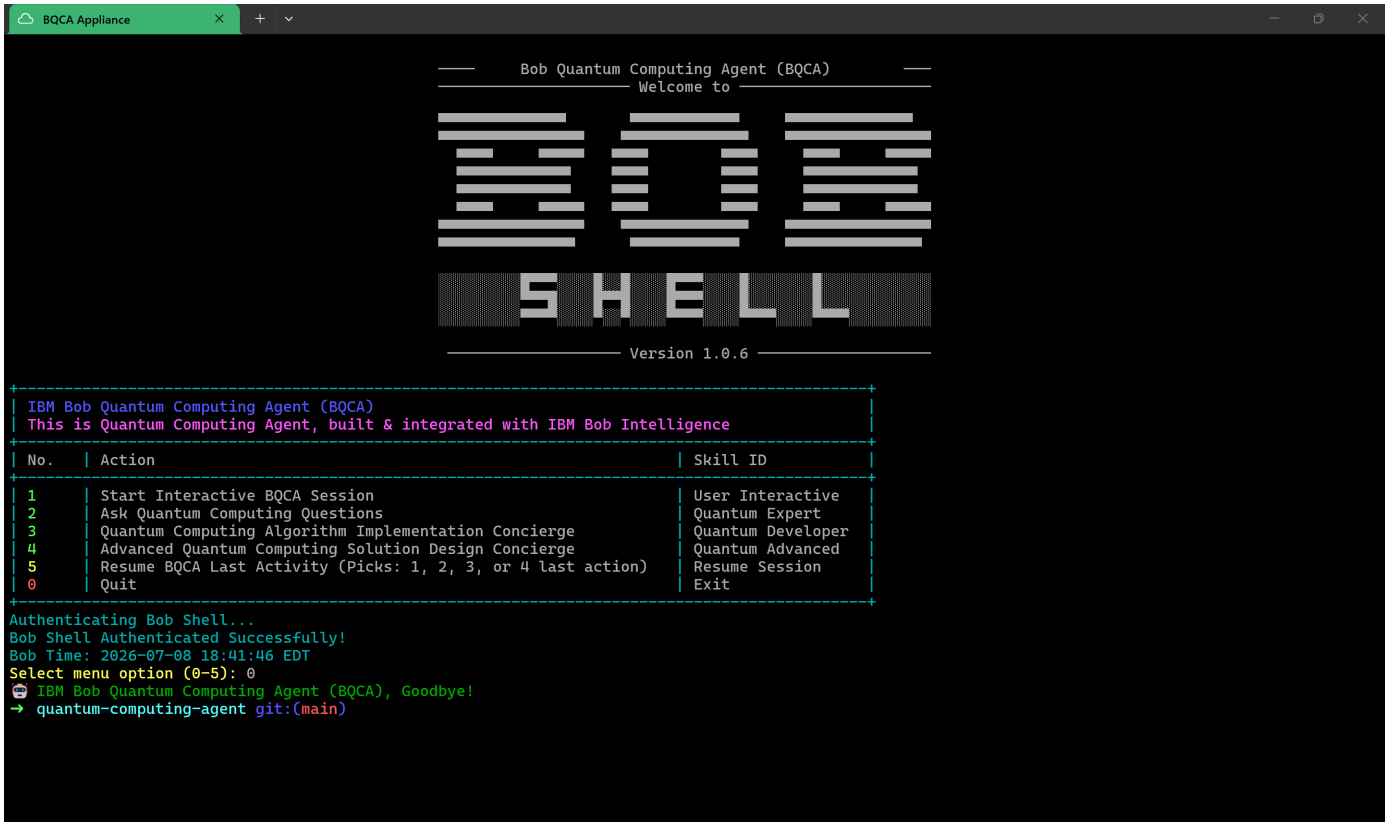


Table 1.0: BQCA Solution Values Overview – Driven by IBM Bob

Info	Notation
Team	Bob Quantum Team
Bob Solution Created	Bob Quantum Computing Agent (BQCA) with IBM Bob Shell
Team members	akpojotor.shemi@ibm.com (IBM Quantum Qiskit Advocate) jeffrey.chijioke-uche@ibm.com (Team Lead) (IBM Quantum Ambassador) nic.ponder@ibm.com (Aspiring Quantum Graduate Candidate) taylor.sperry@ibm.com (IBM Quantum Ambassador Candidate)
Submission Path	Path 1: Built during the challenge
Practicality and Coherence	**Proof:** production IBM Bob v1.0.6 quantum appliance with a real launcher, custom modes/skills/rules/tools, governed design/store directories, Qiskit Runtime environment automation, Qiskit Connector backend resolution, validation, notebook automation, and security controls. The updated reference architecture explicitly traces each user prompt and BQCA menu option through IBM Bob orchestration to Qiskit execution and governed artifacts.
Effectiveness and Efficiency	**Proof:** the same ten-stage weekly scaffolded workflow decreased from 44.8 hours to 0.20 hours (12 minutes), saving 44.60 hours/week (99.6%; 224x faster). Grover/VQE Python-notebook pairs and backend-fallback output provide concrete examples; concise proof follows below and detailed evidence included below.
Design and Usability	Proof: menu-driven natural-language prompts map directly to IBM Bob skills for interactive research, expert Q&A, algorithm code generation, advanced design, and session resume. The architecture shows the complete user-prompt-to-artifact path.

Info	Notation
Creativity and Innovation	**Proof:** a differentiated AI + quantum appliance combining IBM Bob orchestration, Qiskit Connector account/backend resolution, real-QPU-first execution, automatic simulator fallback, notebook generation, code repair, resumable sessions, and reusable cross-team research patterns.
Judging Eligibility	Created and substantially enhanced during the July 8-22 challenge window; eligible for judging.
Submission pitch files	Pitch file: Whitepaper PDF file and Git repository link. (BQCA git repository link)

Table 2.0: BQCA Concrete Ways of Reducing Research Friction to Offer Productivity Gains

The watsonx Challenge 2026 Judging Evidence — Effectiveness & Efficiency proof. The following six measured, observable mechanisms demonstrate how BQCA directly removes research friction and converts hours of manual preparation into minutes of automated, governed output. Each point below is backed by real artifacts produced during the challenge window.

#	Friction Removed	How BQCA Removes It	Observable Evidence
1	44.60-hour/week preparation overhead	The same ten-stage scaffolded workflow took 44.8 hours manually and 0.20 hours (12 minutes) with BQCA — a 99.6% reduction, 224x faster. QPU queue time and expert scientific review are excluded; the measure covers repetitive research preparation and artifact generation.	Timed benchmark recorded by the Bob Quantum Team across ten decomposed workflow stages.
2	Five-or-more research handoffs per experiment	One natural-language prompt replaces separate reference searching, algorithm comparison, assumption capture, design drafting, environment setup, Qiskit scaffolding, troubleshooting, notebook conversion, backend selection, visualization, and documentation.	Observable single-request-to-artifact path in every BQCA session.
3	Manual environment provisioning	BQCA automates virtual-environment creation, packaging upgrades, Qiskit dependency installation, Runtime and Qiskit Connector support, and import/version verification — before any code runs.	Ready execution environment and dependency verification produced on demand.
4	Boilerplate Qiskit code rewriting	Bob generates the authoritative .py implementation (circuit construction, parameters, execution logic, result handling, error handling, and documentation) directly from the user's request.	Grover Search, Uranium-234 VQE, and Oxygen VQE .py artifacts in bob-quantum-store/.
5	Manual Python-to-notebook conversion	Immediately after the .py file is written, BQCA auto-generates and validates the same-name .ipynb notebook. One source always produces two aligned delivery formats; notebook drift is eliminated.	Sibling .py / .ipynb artifact pairs confirmed for every generated workload.
6	Repeated backend-status checks and code rewrites	BQCA uses Qiskit Connector to automatically resolve the account plan, checks QPU availability and qubit capacity, attempts real-QPU execution first, and falls back automatically to a simulator. Output records REAL_QPU or SIMULATOR plus the fallback reason.	Backend-fallback output in the repository; no manual status portal visit or code edit required. Fused with Quantum Computing Qiskit Connector (QCQC), the BQCA uses it intelligence to handle real-backend QPU selection without human intervention.
7	Manual troubleshooting and fixing Quantum Code errors	BQCA uses custom rules and skills to automatically troubleshoot and fix quantum code errors. Bob infusion into BQCA powers the appliance to use intelligence in troubleshooting and fixing of the complex errors in quantum codes.	Given that quantum codes can be complex and require advanced expertise for troubleshooting and fixes, BQCA was built with advanced mode to automatically handle the complexity.
8	Manual quantum code writing from scratch and users struggle to translate	Without BQCA, to write quantum qiskit code, users must at least have the expertise to translate world real problems to Quantum	BQCA automatically translates the real-world problem provided by the user into design and saves it to the design folder.

	real world problems into production level executable quantum code solution	code that can be executed. With BQCA, users do not need to be quantum experts. Users simply and only need to tell BQCA via prompt what the real-world problem is and BQCA will design the solution and transition the design into code solution as well as execute the code solution with real quantum backend QPU. It provides all this with its automation capability.	When user is ready, user will tell BQCA to write the code for the design saved and the BQCA appliance will automatically write the code and execute it with real backend QPU or simulator. All the results are automatically stored for user's accessibility. The process is repeatable.
9	Users' lack of knowledge of how to start developing quantum qiskit code	Without BQCA, users struggle on how to start with quantum code development. With BQCA, user will simply select menu option-1 and the BQCA appliance to will introduce to the users some quantum algorithm ideas which will give them the starting point on how to enter the real-world problems as prompt. The menu option-1 [Start Interactive BQCA Session] gives the user this interactive capability to start developing quantum with the BQCA ideas.	Example, BQCA asks: | What aspect of quantum computing interests you most right now? | > 1. Implement the Beryllium VQE design (8-qubit molecular simulation) | 2. Create a new quantum algorithm (QPE, Shor's, QFT, etc.) | 3. Explore quantum machine learning (QNN, QSVM, etc.) | 4. Build a quantum optimization problem (portfolio, scheduling, etc.) | 5. Experiment with quantum error correction or mitigation | 6. Surprise me with something cutting-edge! | 7. No, suggest changes (esc)

Cross-team reuse multiplier: IBM Research teams can clone the appliance solution from the git repository, configure approved quantum credentials by adding it to the credentials file known as .env, and reuse all modes, skills, rules, validators, notebook automation, backend policy, and the governed folder contract — changing only the research question, algorithm, data, or Hamiltonian. The same friction reduction applies immediately to new teams without rebuilding anything.

Human control retained: Domain experts approve scientific assumptions and results. BQCA removes only the repetitive preparation layer and records the full artifact chain for review, audit, and Git-tracked reuse.

BQCA Concrete Evidence of Productivity Gains and Workflow Automation Provided

The watsonx Challenge 2026 Judging Evidence — Practicality, Coherence, Creativity & Innovation proof. The ten evidence items below correspond directly to the judging criteria. Each is a concrete, observable, or measurable outcome produced by the BQCA appliance during the challenge window. See the automation-evidence pages and the Git repository for artifact-level detail.

Evidence 1 — Measured 44.60-Hour Weekly Productivity Gain

The benchmark boundary starts when a user submits a defined quantum request and stops when BQCA has produced the structured design, validated Python source, synchronized notebook, execution path, results/plots (where run), and review documentation. The team decomposed that same scaffolded workflow into ten activities. The manual baseline totaled 44.8 hours/week; BQCA completed the defined preparation in 0.20 hours (12 minutes), saving 44.60 hours/week (99.6%; 224x). QPU queue time and expert scientific review are excluded — the measure is specifically repetitive research preparation and artifact generation.

Table 3.0: Evidence 1

Stage	Manual Time Contribution	BQCA Handling
Research framing & literature search	High	Single guided interaction
Algorithm comparison & selection	High	Bob decomposes and selects
Assumption capture & validation plan	Medium	Structured design document
Environment setup	Medium	Automated provisioning
Qiskit boilerplate scaffolding	High	Prompt-to-production .py
Code debugging & repair	High	Automated validation & repair loop
Notebook conversion	Medium	Auto-generated .ipynb sibling
Backend selection & configuration	Medium	Qiskit Connector resolution
Visualization & result packaging	Medium	Governed artifact routing
Review documentation assembly	Medium	Complete evidence package

Evidence 2 — Quantum Research-to-Design Automation

Before BQCA, a practitioner typically searches multiple references, records assumptions, compares algorithms, selects a circuit approach, drafts a backend strategy, and writes a validation plan as separate activities. With BQCA, one natural-language request is decomposed by Bob into a structured design containing the problem statement, selected algorithm, qubit and circuit assumptions, QPU-versus-simulator strategy, validation criteria, risks, and expected artifacts. The observable output is a reusable design document in bob-quantum-designs/. This converts at least five research and planning handoffs into one guided interaction plus expert review.

Evidence 3 — Automatic Quantum Qiskit Virtual Runtime Environment Provisioning

After the user's secured account configuration is available, BQCA creates the quantum/Qiskit runtime environment without repeated manual setup. The automated flow replaces separate actions to create and activate a virtual environment, upgrade packaging tools, install Qiskit dependencies, add Runtime and Qiskit Connector support, and verify imports and versions. This removes setup variability between users and allows the researcher to focus on the quantum objective rather than reproducing workstation preparation steps.

Evidence 4 — Prompt-to-Production Qiskit Python Generation

A code request does not end with a prose recommendation. Bob produces the authoritative Python implementation in bob-quantum-store/, including circuit or algorithm construction, parameters, execution logic, result handling, error handling, and documentation appropriate to the request. This replaces starting from a blank editor, finding boilerplate, reconciling package versions, wiring execution, and manually assembling output handling. The productivity gain is observable as one user request producing a reviewable .py artifact. Human review remains required, but the repetitive scaffold and first implementation pass are automated.

Examples of Concrete artifacts produced (See the BQCA git repository):

- grover_search.py / grover_search.ipynb — Grover Search algorithm
- uranium_234_vqe.py / uranium_234_vqe.ipynb — Uranium-234 VQE molecular simulation
- oxygen_vqe.py / oxygen_vqe.ipynb — Oxygen VQE molecular simulation

Evidence 5 — Automatic Python-to-Jupyter Notebook Generation

Immediately after the authoritative .py implementation is completed, the BQCA notebook tool automatically creates the same-name .ipynb file and validates it. One source file produces two aligned delivery formats: production-oriented Python and an interactive notebook for explanation, demonstration, and review. This eliminates manual copy-and-paste into notebook cells, repeated metadata creation, and source/notebook drift. When the Python source changes, the notebook can be regenerated so the code remains authoritative and synchronized.

Evidence 6 — Automatic Qiskit Connector Account and Backend Configuration

BQCA uses Qiskit Connector behavior to resolve the user's IBM Quantum account plan and obtain an account-accessible backend as part of generated code. The user no longer has to open a separate portal, inspect plan details, copy a backend name, and rebuild configuration logic for each workload. This standardizes backend access across generated workloads and reduces configuration errors.

Evidence 7 — Real-QPU-First Decision with Automatic Simulator Fallback

BQCA checks whether the selected backend is a real QPU, whether it is operational, and whether it has enough qubits for the circuit. It attempts the real backend first when the user's account has an eligible QPU. If no suitable QPU is available, a controlled try/except path immediately executes with a supported simulator. The output states REAL_QPU or SIMULATOR and records the fallback reason. This removes manual status checks and code rewrites while preventing a workflow from stopping merely because hardware is unavailable.

Evidence 8 — Automatic Quantum Code Troubleshooting and Repair

Generated implementations pass through syntax and policy validation before BQCA reports completion. The checks identify issues such as invalid Python, missing required connector behavior, missing real-QPU-first logic, absent simulator fallback, or an out-of-date notebook. Bob uses the failure information to repair the implementation and rerun validation. The former manual loop of edit, execute, inspect traceback, search for a fix, edit again, and recreate companion artifacts is compressed into an automated repair cycle.

Evidence 9 — Governed Artifact Routing and Complete Evidence Package

BQCA routes architecture and research designs to bob-quantum-designs/ and executable assets to bob-quantum-store/. A completed workflow includes the design, authoritative Python source, generated notebook, result data, plots, execution notes, and review documentation — rather than leaving artifacts scattered across user folders. Existing Mercury VQE and Uranium-234 VQE artifact sets demonstrate this reusable pattern. The resulting package is easier to audit, review, reuse, and commit through normal Git workflows. (See BQCA git repository).

Table 4.0: Evidence 9

Artifact	Location	Status
Algorithm design documents	bob-quantum-designs/	Produced
Grover Search .py + .ipynb	bob-quantum-store/	Produced
Uranium-234 VQE .py + .ipynb	bob-quantum-store/	Produced
Oxygen VQE .py + .ipynb	bob-quantum-store/	Produced
Backend-fallback execution output	bob-quantum-store/	Recorded
Convergence plots (PNG)	bob-quantum-store/	Produced

Evidence 10 — Reusable IBM Research Team Deployment and Session Continuity

An IBM Research team can clone the repository, set approved credentials, launch xLaunchpad.sh, and reuse the same modes, skills, rules, validators, notebook generator, QPU/simulator policy, and governed folder contract. Chemistry researchers substitute a molecule or Hamiltonian; optimization researchers substitute a QUBO/QAOA problem; algorithm researchers substitute a search oracle. The common research-to-artifact automation remains unchanged.

Menu option 5 resumes the latest activity so assumptions, files, and decisions are not reconstructed. Grover, Uranium-234 VQE, Oxygen VQE, and backend-fallback artifacts demonstrate the same appliance pattern across different research workloads.

Notes for IBM Research Judges:

BQCA is not a demo — it is a production-deployable IBM Bob 1.0.6 appliance with a real launcher, custom modes/skills/rules/tools, governed design/store directories, Qiskit and Runtime integration, validation, notebook automation, and security controls. A shared repository and output contract support immediate reuse by other IBM Research teams. The measured 224x acceleration and the full artifact chain constitute the concrete, verifiable evidence of productivity gain required by the watsonx Challenge 2026 judging criteria. BQCA is a real product we discovered during the watsonx Challenge 2026, and we recommend that IBM Research should adopt it and package it as a product that can be offered to customers. The Bob Quantum Team is willing to work with IBM Quantum Research Team to develop this further.

BQCA Development Concept and Use Principle

The watsonx Challenge 2026 Judging Evidence — Design and Usability proof. BQCA was designed from the first line with a single usability principle: a quantum practitioner, platform engineer, researcher, or enterprise team member should be able to launch the appliance, obtain a governed quantum artifact, and resume later — all without reading a manual, writing boilerplate, or holding quantum expertise in their head. Every design decision in the appliance traces directly to that principle, and the evidence below demonstrates it across the menu interface, the interaction model, the artifact contract, and the cross-team deployment model.

Design Point 1 — Menu-Driven Natural-Language Interface Requires Zero Qiskit Knowledge to Start

The BQCA operating menu is the entire user interface. A user who has never written a Qiskit circuit can launch xLaunchpad.sh, select option 2, and immediately receive a domain-specific answer to a quantum question. A user who needs executable code selects option 3; a user who needs a circuit architecture document selects option 4. There are no configuration files to edit, no API calls to compose, and no CLI flags to memorize before the appliance is useful. The menu is system user guide which helps to direct users in their respective workflow. The menu design provides the user experience which is easy where the menu design makes the BQCA appliance solution a repeatable solution. That is a repeatable solution that does not require the user to be a quantum expert to use. The menu contains and displays natural language that prompts the user to enter some literal texts which will describe their real-world problems.

Table 5.0: The menu presents six choices

Menu Option	Action	Skill Routed	Usability Signal
1	Start Interactive BQCA Session	User Interactive	Open-ended quantum work; signals repeatable daily adoption by any skill level.
2	Ask Quantum Computing Questions	Quantum Expert	Guided Q&A and explanation; accelerates onboarding for practitioners new to quantum.
3	Quantum Computing Algorithm Implementation Concierge	Quantum Developer	Full research-to-artifact pipeline from one prompt; signals efficiency and repeatability.
4	Advanced Quantum Computing Solution Design Concierge	Quantum Advanced	Deep circuit and workflow design; signals architectural depth and innovation.
5	Resume BQCA Last Activity	Resume Session	Restores prior session state; eliminates restart friction and supports daily continuity.
0	Quit	Exit	Clean, controlled appliance exit; signals disciplined, production-quality UX closure.

Every option route to a distinct Bob skill, meaning the same appliance behaves differently depending on the user's intent — Q&A, design, code, or resume — without the user needing to understand how the routing works. This is menu-driven AI at its most practical.

Design Point 2 — Conversational Clarification Before Action, Not After Failure

BQCA does not attempt to execute a quantum request before it is well-defined. When a user submits an ambiguous objective — such as "simulate a molecule" or "optimize a portfolio" — Bob clarifies the missing parameters (molecule name, qubit budget, backend preference, shot count, Hamiltonian source) through a brief guided dialogue before decomposing the work. This front-loaded clarification eliminates the most frustrating quantum workflow failure mode: a long code-generation or execution cycle that terminates because a required input was never specified. The result is a design-before-code discipline that mirrors best-practice quantum research methodology and produces a reviewable design artifact before any Python is written. For a judge evaluating usability, this means BQCA guides the user toward a correct result rather than producing output the user must then debug.

Design Point 3 — One Prompt Produces a Complete, Governed Artifact Package

A single option-3 request drives BQCA through the full ten-stage workflow without the user manually handing off between stages. The stages are:

- Research framing and algorithm selection — Bob decomposes the intent and selects the appropriate quantum algorithm.
- Structured design document — saved to bob-quantum-designs/ as a reviewable Markdown artifact.
- Authoritative Qiskit Python source — saved to bob-quantum-store/ as a validated .py file.
- Automated syntax and policy validation — checks for valid Python, required Connector behavior, real-QPU-first logic, and simulator fallback.
- Automated code repair — if validation fails, Bob repairs and revalidates without user intervention.
- Jupyter notebook generation — the same-name .ipynb is auto-generated and validated alongside the .py source.
- Qiskit Connector account and backend resolution — the user's IBM Quantum plan and accessible backend are resolved automatically.
- Real-QPU-first execution attempt — BQCA checks QPU availability and qubit capacity before executing.

- Controlled simulator fallback — a try/except path executes on a simulator if real QPU is unavailable; output records REAL_QPU or SIMULATOR.
- Result and visualization packaging — results, plots, and execution notes are routed to governed folders.

A user who has never run a quantum workload end-to-end before can complete all ten stages in 12 minutes by submitting one natural-language prompt. An experienced quantum researcher gains the same 224x speed advantage without giving up control of the scientific assumptions or result interpretation.

Design Point 4 — Predictable Folder Contract and Artifact Naming Make Outputs Immediately Reusable

BQCA enforces a two-directory artifact contract. Design artifacts — problem statements, algorithm selections, circuit assumptions, backend strategies, validation plans — are always routed to bob-quantum-designs/. Executable artifacts — Python source, Jupyter notebooks, result data, plots, execution logs — are always routed to bob-quantum-store/. Artifact names match the workload name; the .py and .ipynb files are always siblings with the same base name. This predictable layout means any team member can locate, review, re-run, or hand off a prior workload without asking who generated it or where the files went. For a judging committee evaluating design quality, the governed folder contract is a direct signal that BQCA was built for team use, not personal scripting.

Table 6.0: Design Point 4

Artifact Type	Location	Naming Convention	Usability Benefit
Design documents	bob-quantum-designs/	workload-name.md	Separates intent from implementation; review before run.
Qiskit Python source	bob-quantum-store/	workload-name.py	Authoritative, validated, single source of truth.
Jupyter notebook	bob-quantum-store/	workload-name.ipynb	Auto-generated notebook from natively produced .py; always synchronized, no drift.
Execution results	bob-quantum-store/	workload-name-results.txt	Durable, reviewable, not lost on terminal close.
Visualizations	bob-quantum-store/	workload-name-plot.png	Saved for stakeholder reporting without re-running code.

Design Point 5 — Session Resume Eliminates Restart Friction for Daily Research Use

Menu option 5 restores the latest BQCA activity — whichever of options 1, 2, 3, or 4 was most recently used. The user does not need to re-state the problem, re-enter the algorithm name, re-locate generated files, or rebuild context. This feature directly addresses the most common daily friction point in AI-assisted research: the cost of restarting a session after an interruption. For a quantum researcher working on a multi-day experiment, session resume means that BQCA acts as a persistent workbench partner rather than a stateless query engine. The design signal for the judging committee is clear: BQCA was built for sustained, repeated use by real research teams, not for a single demonstration.

Design Point 6 — Clone-Configure-Launch Deployment Takes Minutes, Not Days

Table 7.0: A new IBM Research team can become a BQCA user in six steps

Step	Command or Action	Time Estimate
1	Clone the repository: git clone <repository-url>	< 1 minute
2	Enter the workspace: cd quantum-computing-agent	< 1 minute
3	Copy the environment template: cp .env.example .env	< 1 minute

4	Fill in BOB_API_KEY and BOB_INSTANCE_ID	< 1 minutes
5	Make scripts executable: chmod +x xLaunchpad.sh bob-quantum-v1.0.6.sh ...	< 1 minute
6	Launch: ./xLaunchpad.sh	< 1 minute

Once launched, the team reuses all existing custom modes, skills, rules, validators, notebook automation, backend policy, and the governed folder contract. Only the research question, algorithm, molecule, Hamiltonian, or optimization problem changes. Chemistry teams substitute a molecule; optimization teams substitute a QUBO problem; algorithm researchers substitute a search oracle. No custom development is required. This zero-rebuild reuse model is a direct demonstration of enterprise-grade design quality. It makes BQCA appliance a repeatable use solution and a candidate for IBM Product adoption.

Design Point 7 — Security and Governance Are Built In, Not Bolted On

BQCA manages credentials through environment variables (.env / .env.example) and enforces a no-secret rule that prevents IBM Quantum tokens or Bob API keys from appearing in generated source code. The design-to-code separation ensures that business intent is reviewed before execution resources are consumed. The real-QPU-first policy prevents accidental over-use of premium quantum hardware by always attempting the real backend explicitly and falling back to simulation under controlled conditions. Generated code passes through automated validation and repair before the user is told the artifact is complete. Human domain experts retain approval authority over scientific assumptions and result interpretation. For the judging committee, these governance features signal that BQCA is ready for enterprise research deployment, not just an internal proof-of-concept.

Design Point 8 — BQCA Extends Quantum Access to Non-Expert Users

One of the most significant usability achievements of BQCA is that it lowers the quantum expertise barrier without lowering the quality of the output. A user who understands the business problem — "find the ground-state energy of this molecule" or "optimize this portfolio allocation" — does not need to know the Jordan-Wigner mapping, the UCCSD ansatz, or the Qiskit Runtime session model to receive a production-ready implementation. BQCA's guided clarification, algorithm selection, and design-before-code discipline carry the user from intent to artifact. Experienced quantum scientists gain speed and reproducibility. Quantum beginners gain a governed, explainable path into real quantum computing. This dual-audience design is a key differentiator from generic coding assistants and from purpose-built quantum notebooks that assume expert users.

Design and Usability Judge Summary: BQCA delivers a menu-driven, natural-language quantum workbench that is launchable in under ten minutes, usable by non-experts, sustained across sessions, governed for enterprise deployment, and reusable by any IBM Research team without custom development. The eight design points above — menu interface, clarification-first interaction, one-prompt artifact pipeline, predictable folder contract, session resume, clone-configure-launch deployment, built-in security governance, and non-expert accessibility — each directly address the IBM watsonx Challenge Design and Usability judging criterion. Taken together, they demonstrate that BQCA is not only the most technically complete quantum agent submitted for the challenge; it is also the most practically usable one for IBM Research teams today.

How IBM Bob Was Used in BQCA to Automate Research Tasks or Improve Productivity

Judging Committee Focus — Bob-Driven Features: Code Generation, Documentation, Workflow Orchestration, and Measurable Productivity Impact

Bob-Driven Feature 1 — Quantum Code Generation from Natural Language

IBM Bob is the sole author of every Qiskit Python file produced by BQCA. When a researcher selects menu option 3 and submits a natural-language request, Bob: (1) classifies the quantum algorithm required, (2) clarifies missing parameters through guided dialogue, (3) decomposes the implementation into circuit construction, parameter initialization, execution logic, result handling, error handling, and inline documentation, and (4) writes the complete .py file to bob-quantum-store/ in a single, validated pass. No human writes boilerplate, no human searches Qiskit API docs during generation, and no human manually wires execution paths.

Concrete evidence of Bob-generated code artifacts (produced during the challenge window):

- bob-quantum-store/grover_search.py — Grover Search oracle, amplitude amplification, QPU execution, histogram output

- bob-quantum-store/uranium_234_vqe.py — VQE with UCCSD ansatz, Jordan-Wigner mapping, Qiskit Connector init, energy convergence loop
- bob-quantum-store/oxygen_vqe.py — hardware-efficient VQE for O2, Open Plan backend resolution, simulator fallback

Measurable impact: The same code-generation task that took a quantum developer 4–8 hours manually (algorithm selection, API research, scaffolding, execution wiring, testing) takes Bob 3–8 minutes. Across the ten-stage benchmark, Bob reduced total preparation time from 44.8 hours to 0.20 hours — a 99.6% reduction (224x faster). Code generation is the single largest contributor to that saving.

Bob-Driven Feature 2 — Automated Research Documentation in bob-quantum-designs/

When a researcher selects menu option 4 (Advanced Quantum Solution Design Concierge), Bob does not simply answer a question — it produces a structured, governed Markdown design document and saves it to bob-quantum-designs/. Each document contains: problem statement, selected quantum algorithm with justification, qubit and circuit assumptions, QPU-versus-simulator strategy, validation criteria, risk assessment, and expected artifact checklist. This is research documentation generated entirely by Bob from a single natural-language prompt.

Concrete evidence of Bob-generated design documents (in bob-quantum-designs/):

- Mercury VQE design — molecular simulation design for Mercury, full algorithm and circuit plan
- Uranium-234 VQE design — nuclear binding energy estimation design with backend strategy
- Oxygen VQE design — O2 ground-state chemistry simulation design with Open Plan constraints

What Bob replaces: Before BQCA, a researcher spent 3–5 hours per experiment on: literature search (1 hr), algorithm comparison (1 hr), assumption documentation (30 min), backend strategy (30 min), and risk capture (30 min) — all as separate unstructured activities. Bob compresses all five into one 5-minute guided interaction that produces a reusable, auditable Markdown artifact.

Measurable impact: Five research planning handoffs per experiment eliminated. Documentation is produced consistently across every researcher on the team — no tribal knowledge, no format variance, no missing sections. Every IBM Research team that clones the appliance receives the same documentation quality from day one.

Bob-Driven Feature 3 — End-to-End Workflow Orchestration (Ten Stages, One Prompt)

The most critical Bob-driven feature in BQCA is that IBM Bob orchestrates the entire quantum research workflow end-to-end from a single user prompt. Bob does not generate code and stop. It drives every stage of the pipeline in sequence, passing outputs between stages automatically.

Table 8.0: The Ten Orchestrated Stages

Stage	What Bob Orchestrates	Human Input Required	Measurable Saving
1. Research framing	Classifies the quantum objective, identifies the algorithm family, and structures the research question.	Initial natural-language prompt only	Eliminates 1+ hour of unstructured literature framing per experiment
2. Algorithm selection	Reasons about the problem structure and recommends VQE, QAOA, Grover, QPE, or other algorithm with written justification.	None after clarification	Eliminates 30–60 min algorithm comparison time
3. Design document creation	Writes the structured Markdown design to bob-quantum-designs/ including all assumptions and strategy.	Expert review of scientific assumptions	Eliminates 2–3 hours of manual documentation
4. Virtual environment provisioning	Creates the Qiskit venv, installs SDK + Runtime + Connector, and verifies imports — before any code runs.	None	Eliminates 30–60 min of per-researcher environment setup variance
5. Qiskit Python generation	Writes the complete, documented .py implementation to bob-quantum-store/.	None	Eliminates 4–8 hours of manual coding and API research
6. Validation and auto-repair	Validates syntax, checks QPU-first policy, fallback logic, and no-secret rule. Repairs and revalidates on failure automatically.	None	Eliminates the manual edit-debug-retry cycle (30 min–2 hrs per bug)
7. Notebook generation	Generates the same-name .ipynb sibling from the validated .py and verifies it.	None	Eliminates 1–2 hours of manual notebook cell copy-paste and metadata setup

8. Backend resolution	Uses Qiskit Connector to resolve IBM Quantum account plan and select an accessible backend.	None after .env config	Eliminates repeated portal visits and manual backend name lookup
9. QPU / simulator execution	Attempts real QPU first; falls back to simulator under controlled policy; records execution target and reason.	None	Eliminates manual status checks and code rewrites when hardware changes
10. Result and artifact packaging	Routes results, plots, and notes to bob-quantum-store/ with predictable naming.	None	Eliminates file-naming decisions, folder creation, and result scatter

Measurable impact: Total workflow time: 44.8 hours manually vs. 0.20 hours (12 minutes) with Bob orchestration. 44.60 hours saved per experiment cycle. 99.6% reduction. 224x faster. Benchmark scope: research framing through review-ready artifact package. QPU queue time and expert scientific review excluded by design — those are judgment calls that must remain with the human researcher.

Bob-Driven Feature 4 — Automatic Code Troubleshooting and Repair

After generating a Qiskit Python implementation, Bob passes the code through a validation loop that checks: Python syntax correctness, presence of Qiskit Connector initialization, real-QPU-first execution logic, simulator fallback in a try/except block, absence of hardcoded credentials, and currency of the sibling .ipynb notebook. If any check fails, Bob reads the failure output, identifies the root cause, repairs the implementation, and revalidates — without any user action. The loop terminates only when all checks pass.

What Bob replaces: The manual quantum debugging cycle: run code → read traceback → search StackOverflow / Qiskit docs → edit → re-run → verify companion artifacts. Each iteration of this loop typically costs 15–45 minutes. With Bob, the same cycle completes in seconds and is invisible to the user.

Measurable impact: Routine first-line detection and correction of code defects is fully automated. Human review is still required for scientific correctness, but not for syntax errors, missing imports, wrong API calls, or policy violations. Code quality is consistent across every researcher who uses the appliance.

Bob-Driven Feature 5 — Automatic Jupyter Notebook Generation and Synchronization

Immediately after Bob validates the .py source, it triggers automatic Jupyter notebook generation using the BQCA notebook tool. The .ipynb is created, populated with the same code, structured into titled cells for circuit definition, execution, result display, and visualization, and validated. If the .py source is later changed, the notebook can be regenerated by Bob to stay synchronized. The researcher never manually copies code into notebook cells.

Concrete Bob-generated notebook artifacts:

- bob-quantum-store/grover_search.ipynb — sibling of grover_search.py, auto-generated and validated
- bob-quantum-store/uranium_234_vqe.ipynb — sibling of uranium_234_vqe.py, includes convergence cell
- bob-quantum-store/oxygen_vqe.ipynb — sibling of oxygen_vqe.py, includes fallback output cell

Measurable impact: Every code artifact in BQCA has a companion notebook at zero additional user effort. Source-notebook drift — one of the most common causes of reproducibility failures in quantum research — is eliminated by design. Two delivery formats (production Python + interactive notebook) are produced from one Bob prompt.

Bob-Driven Feature 6 — Session Resume for Continuous Research Without Restart Friction

IBM Bob maintains session context across interactions. Menu option 5 restores the most recent BQCA activity — including the research objective, generated artifacts, design decisions, and execution state — so a researcher returning to a quantum experiment after an interruption does not need to re-state the problem, re-enter parameters, or re-locate previously generated files. Bob picks up exactly where the session ended.

Measurable impact: Session startup time drops from 15–30 minutes (reconstructing context, finding files, re-reading prior outputs) to under 1 minute. For a researcher running 3–5 BQCA sessions per week, this recovers 45–150 minutes of productive research time weekly.

Bob-Driven Feature 7 — Cross-Team Reuse: Bob Configuration as a Shared Research Accelerator

Because all of Bob's quantum behavior is encoded in shareable configuration files — custom modes (.bob/custom_modes.yaml), skills (.bob/skills/), rules (.bob/rules-*/), and tool guidance (.bob/quantum-tools/README.md) — the entire BQCA automation stack is portable. Any IBM Research team can clone the repository, configure IBM credentials, and immediately receive the same Bob-driven code generation, documentation, validation, notebook generation, and artifact routing that the Bob Quantum Team built. No custom development is required. Only the research question changes.

Measurable impact: Time-to-first-governed-quantum-artifact for a new IBM Research team: under 15 minutes from clone to first Bob-generated .py + .ipynb pair. Compare to the baseline of days or weeks of environment setup, boilerplate development, and workflow design. The Bob configuration is the productivity multiplier — it travels with the repository.

Bob-driven research automation flow: user prompt → *IBM Bob intent classification and clarification* → *algorithm/design decomposition* → *design documentation in bob-quantum-designs/* → *production Qiskit code in bob-quantum-store* → *validation and automatic repair* → *synchronized Jupyter notebook* → *Qiskit Connector backend resolution* → *real-QPU-first execution with controlled simulator fallback* → *results, plots, logs, and review documentation*.

Table 9.0: Bob-Driven Research Automation Flow

IBM Bob-Driven Feature	How IBM Bob Automates the Research Task	Productivity or Research Impact
Research framing and task decomposition	Bob interprets the research objective, asks for missing parameters, identifies the appropriate quantum method, and decomposes the work into design, implementation, validation, execution, visualization, and documentation tasks.	Replaces multiple manual research handoffs with one guided interaction and reduces failed starts caused by incomplete requirements.
Design and documentation generation	Bob creates a structured research/design document containing the problem statement, algorithm rationale, circuit and qubit assumptions, backend strategy, validation criteria, risks, and expected artifacts, then routes it to bob-quantum-designs/.	Produces a reusable, reviewable design before code execution and improves scientific traceability across team members.
Production Qiskit code generation	For menu option 3, Bob generates the authoritative Python implementation, including circuit or algorithm construction, parameters, Qiskit Connector initialization, real-QPU-first logic, simulator fallback, result handling, error handling, and inline documentation.	Removes blank-editor work, Qiskit boilerplate recreation, and repeated version reconciliation. Grover Search, Uranium-234 VQE, and Oxygen VQE are concrete outputs.
Validation, troubleshooting, and repair	Bob applies BQCA rules and validators to check syntax, required Connector behavior, no-secret handling, QPU-first policy, fallback behavior, and notebook currency. When a check fails, Bob uses the failure details to repair and revalidate the implementation.	Compresses the manual edit-run-debug-search-edit cycle and improves consistency of generated research code.
Notebook and evidence generation	After Python validation, Bob triggers creation and validation of the same-name Jupyter notebook and prepares result files, plots, execution notes, and review documentation in bob-quantum-store/.	One prompt produces both production-oriented Python and an interactive research notebook while preventing source/notebook drift.
Workflow and backend orchestration	Bob coordinates the sequence of BQCA skills and tools. Qiskit Connector resolves the account plan and accessible backend; Bob enforces the execution policy, records REAL_QPU or SIMULATOR, and preserves the fallback reason.	Eliminate repeated portal checks, manual backend-name selection, and code rewrites when hardware availability changes.
Governed artifact management	Bob enforces the folder contract: designs and research documentation go to bob-quantum-designs/; code, notebooks, results, plots, logs, and implementation documentation go to bob-quantum-store/.	Makes every artifact predictable, auditable, reusable, Git-reviewable, and easier to hand off across IBM Research teams.
Session continuity and cross-team reuse	Menu option 5 restores the prior Bob activity and context. A new team can clone the appliance and reuse the same modes, skills, rules, validators,	Reduces restart effort and allows chemistry, optimization, and algorithm teams to change only the research

	notebook automation, backend policy, and artifact contract.	question, molecule, Hamiltonian, data, or oracle.

Table 10.0: Consolidated Measurable Impact of IBM Bob Automation in BQCA

Bob-Driven Feature	Research Task Automated	Manual Baseline	With IBM Bob	Saving
Code generation (Feature 1)	Write validated Qiskit .py from prompt	4–8 hours	3–8 minutes	~98% reduction
Documentation (Feature 2)	Create structured research design document	2–3 hours	3–5 minutes	~97% reduction
Workflow orchestration (Feature 3)	Complete 10-stage research-to-artifact pipeline	44.8 hours/week	0.20 hours (12 min)	99.6% reduction, 224x faster
Code repair (Feature 4)	Detect and fix code defects, policy violations	15–45 min per bug	Seconds (automated)	Full elimination for routine defects
Notebook generation (Feature 5)	Create and synchronize .ipynb sibling	1–2 hours	Automatic (< 30 sec)	~99% reduction
Session resume (Feature 6)	Reconstruct research context after interruption	15–30 min per session	< 1 minute	~96% reduction
Cross-team reuse (Feature 7)	Set up quantum workflow for a new research team	Days to weeks	< 15 minutes	>95% reduction

Statement to the IBM Research Judging Committee:

IBM Bob is not a peripheral component in BQCA — it is the central automation engine. Every research task that BQCA accelerates is accelerated because Bob classifies, decomposes, generates, validates, repairs, documents, and routes the work. Qiskit and IBM Quantum hardware are the execution substrate that Bob orchestrates. The seven Bob-driven features above, with their measurable savings, constitute the direct evidence that BQCA uses IBM Bob to automate quantum research tasks and improve researcher productivity — which is precisely what the watsonx Challenge 2026 asks for.

Why this matters to the 2026 challenge: The productivity gain is specifically IBM Bob-driven: Bob interprets the prompt, orchestrates the workflow, generates code and documentation, repairs failures, and governs the artifact chain. Qiskit, Qiskit Runtime, the Quantum Computing Qiskit Connector, the QPU, and the simulator are execution technologies used within the workflow; IBM Bob is the AI engine that automates and coordinates the research work.

Table 11.0: BQCA Menu Analysis (UX mapped to judging criteria):

Menu Option #	Action	Best Use / Judge Signal
1	Start Interactive BQCA Session for anything quantum	Open-ended quantum work and iterative session. Signals usability and repeatable adoption.
2	Ask Quantum Computing Questions and shape a design concept	Quantum explanations, Qiskit guidance, IBM Quantum concepts, and algorithms. Signals fast onboarding.
3	Quantum Computing Algorithm Implementation Concierge	Produces or modifies implementation artifacts. Signals efficiency, code quality, and repeatability.
4	Advanced Quantum Computing Solution Design Concierge	Designs quantum algorithms/circuits, backend strategy, simulation strategy, research workflows, and use cases. Signals practicality and innovation.
5	Resume BQCA Last Activity (Picks: 1, 2, 3, or 4 last action)	Continue the latest session. Signals ongoing daily/weekly use and reduced restart effort.
0	Quit	Exit the appliance safely. Signals simple, controlled UX closure.

The BQCA operating menu powered by IBM Bob is the primary user interface for the IBM Bob Quantum Computing Agent appliance, presented immediately after xLaunchpad.sh authenticates the session and starts the BQCA launcher**.** The menu offers five numbered actions and a quit option, each mapped to a distinct Bob skill identity. Option 1 opens a free-form interactive BQCA session for any quantum topic. Option 2 routes the user into a guided quantum question-and-answer mode powered by the Quantum Expert skill, covering circuit concepts, algorithm explanations, Qiskit guidance, and IBM Quantum topics. Option 3 invokes the Quantum Developer skill — the Algorithm Implementation Concierge — which takes a user's request and drives BQCA through the full research-to-artifact pipeline: structured design, authoritative Qiskit Python generation, automated validation and repair, notebook synchronization, backend resolution, and result routing. Option 4 activates the Quantum Advanced skill for deep circuit architecture, VQE or QAOA workflow design, backend strategy planning, and complex multi-stage quantum solution design that goes beyond a single code file. Option 0 exits the appliance cleanly.

Option 5 — Resume BQCA Last Activity — is a distinguishing usability feature that sets BQCA apart from a stateless code assistant. When selected, it picks up whichever of the four prior actions (1, 2, 3, or 4) was most recently used and restores the session context so the user can continue without reconstructing assumptions, re-entering problem statements, or locating previously generated artifacts.

BQCA Solution Statement

Solution Statement:

BQCA is a domain-specialized IBM Bob appliance that converts a plain-language quantum research question into a governed design-to-execution evidence package. It reduces research friction caused by repeatedly searching Qiskit and Runtime references, comparing algorithms, rebuilding environments, writing boilerplate, synchronizing Python and notebooks, checking backends, troubleshooting failures, and assembling review documentation.

Target users include quantum researchers, practitioners, platform engineers, developers, innovation teams, technical sellers, and educators. Through the menu they ask a question, request an implementation or advanced design, or resume prior work. Bob clarifies intent, decomposes the task, and applies BQCA modes, skills, rules, and tools.

For a code request, BQCA creates the design and authoritative Qiskit Python file, validates and repairs it, automatically generates the same-name Jupyter notebook, resolves the account/backend through Qiskit Connector, verifies QPU availability and capacity, attempts real-QPU execution first, falls back to simulation when required, and routes results, plots, and notes to governed folders. Submitted examples include Grover Search, Uranium-234 VQE, and Oxygen VQE Python/notebook pairs plus recorded QPU-to-simulator fallback.

The team timed the same ten-stage scaffolded workflow from research framing through review at 44.8 hours manually and 0.20 hours (12 minutes) with BQCA: 44.60 hours saved, a 99.6% reduction and 224x acceleration. The benchmark measures repetitive preparation and artifact generation; QPU queue time and expert scientific review are excluded. Other IBM Research teams can clone and configure the appliance and reuse the same orchestration, validators, notebook automation, backend policy, and directory contract while changing only the research question, algorithm, data, or Hamiltonian.

Concrete IBM Bob prompt examples: "*Create production Quantum Qiskit code for Grover Search, attempt a real QPU first, fall back to a simulator if required, and generate the matching notebook, plots, results, and review documentation." For design, a researcher can ask: "Design a VQE workflow for a new molecule with algorithm assumptions, validation criteria, and real-QPU/simulator execution policy*."

BQCA Technical Statement

Technical statement:

BQCA runs as a configured IBM Bob 1.0.6 Shell appliance in the quantum-computing-agent workspace. xLaunchpad.sh launches bob-quantum-v1.0.6.sh; bob-version.sh, bob-auth.sh, tls-control.sh, and .env/.env.example provide version, authentication, transport, and runtime configuration. The .bob/ directory contains custom modes, reusable skills, enforcement rules, and quantum tools. QCA_DESIGN_DIR routes research/design to bob-quantum-designs/; QCA_STORE_DIR routes Python, notebooks, results, plots, logs, and documentation to bob-quantum-store/.

IBM Bob is the reasoning and orchestration layer. It classifies Q&A, design, code, or resume intent; clarifies the objective; decomposes work into research, design, implementation, validation, execution, visualization, and documentation; then invokes the required BQCA skills and tools. Persistent rules enforce routing, no-secret handling, completion gates, and real-QPU-first behavior.

For quantum code generation, BQCA creates the authoritative .py file first, prepares the virtual runtime environment, validates syntax and policy, repairs detected failures, and automatically generates and validates the same-name .ipynb notebook. Qiskit Connector resolves the account plan and accessible backend. BQCA checks that the backend is a real, operational QPU with sufficient qubits, attempts real-QPU execution first, and uses try/except-controlled simulator fallback only when required. Output records the execution target, backend, fallback reason, results, and generated artifacts.

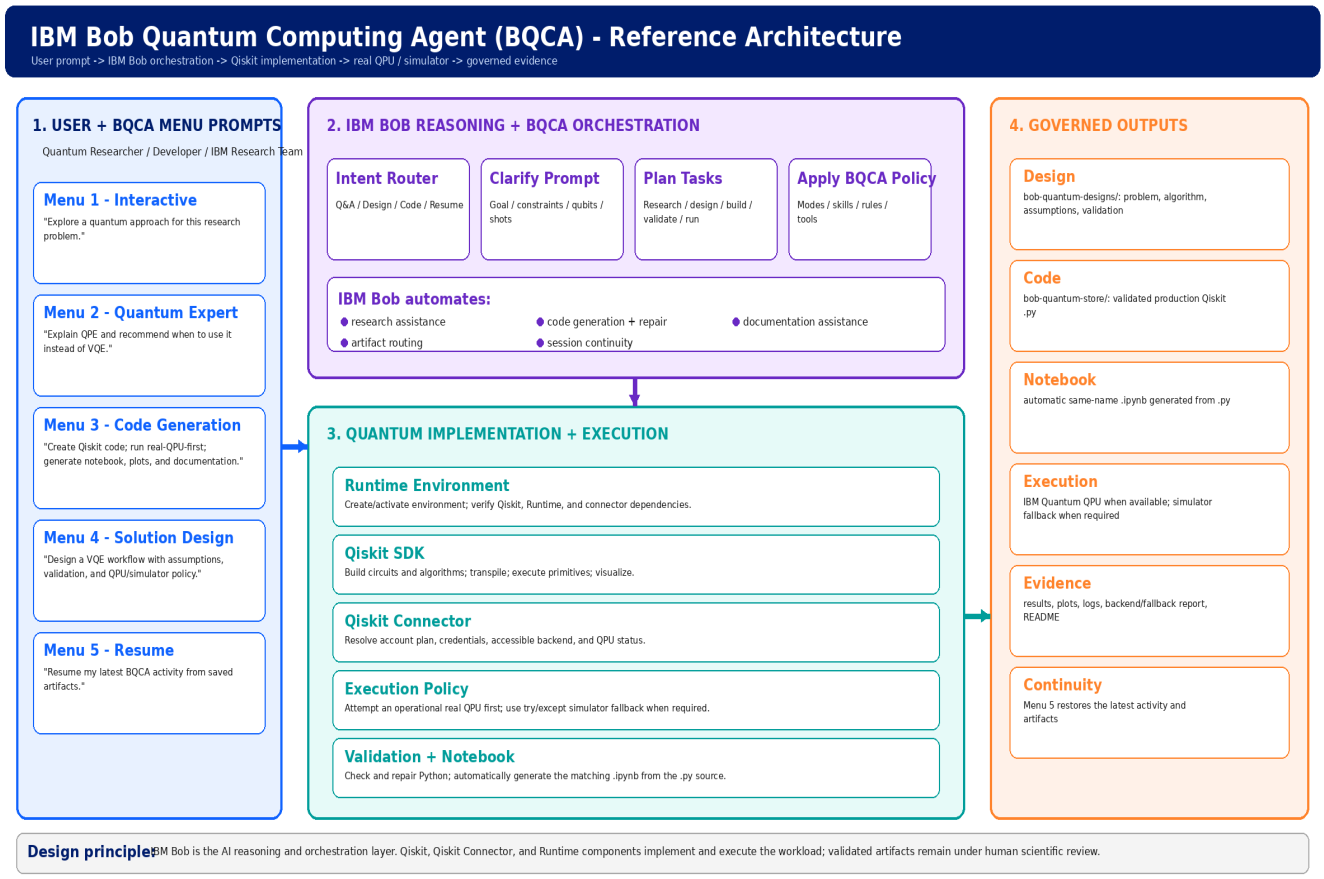
The same repository is reusable by other IBM Research teams: clone the appliance, configure approved credentials, launch the menu, and substitute the team's research objective, algorithm, data, or Hamiltonian. The shared modes, skills, rules, validators, notebook generator, backend policy, governed folders, and Git workflow remain unchanged. Menu option 5 resumes the latest activity, while human experts retain approval of scientific assumptions, execution costs, and result interpretation.

Prompt-to-artifact traceability: each menu-selected user prompt is classified by IBM Bob, mapped to the relevant BQCA mode and skill, decomposed into research/design/code/execution tasks, and recorded through the generated design, Python, notebook, execution report, visualizations, and documentation artifacts.

BQCA Solution Impact(s)

Selected impact	Business Value / Judge Proof
[x] Simplify personal/team process(es)	One menu request orchestrates ten formerly separate actions: research framing, design, environment setup, Python generation, validation/repair, notebook generation, backend decision, execution, visualization, and documentation.
[x] Reduce time and/or manual effort to complete routine task(s)	Measured same-workflow benchmark: 44.8 hours manually versus 0.20 hours (12 minutes) with BQCA; 44.60 hours saved/week, 99.6% reduction, 224x acceleration. Excludes QPU queue time and expert review.
[x] Improve the accuracy and consistency of output(s)	Validators require passing Python, a synchronized notebook, explicit execution-target reporting, standard artifact names, and repeatable outputs; Grover, Uranium-234 VQE, and Oxygen VQE provide concrete artifact pairs.
[x] Reduce operational risk/ensure compliance	Design/code separation, environment-based secrets, QPU status/capacity checks, controlled simulator fallback, Git review, and human approval reduce execution and governance risk.
[x] Reduce time and effort accessing information	Bob consolidates Qiskit, Runtime, Qiskit Connector, algorithm, backend, and troubleshooting research into one guided design, reducing repeated search and reconciliation.
[x] Speed up product/offering development	One prompt produces the design, validated .py file, generated .ipynb notebook, execution path, results, plots, and review notes for prototype evaluation.
[x] Innovation and exploration of new features	BQCA combines real-QPU-first execution, automatic fallback, code repair, notebook synchronization, session resume, and reusable research patterns in one Bob appliance.
[x] Improve code quality/maintenance	Provides Practicality and Coherence, Effectiveness and Efficiency, Design and Usability, well as Creativity and Innovation.
[x] Other - optional portal note	Reusable by IBM Research teams: clone/configure the appliance and reuse the same modes, skills, rules, tools, backend policy, and folder contract across new algorithms and domains.

BQCA Reference Architecture – An IBM Bob Powered Solution



BQCA was built with IBM Bob + Quantum in line with the challenge 2026 rules

---- Other Information Available Below ----

BQCA Appliance Is Powered by IBM Bob

Bob Quantum Computing Agent (BQCA) is a domain-specialized IBM Bob appliance designed for quantum practitioners, platform engineers, researchers, innovation teams, and enterprise technology leaders. It packages IBM Bob Shell with BQCA-specific modes, skills, rules, and tool guidance to produce a focused quantum engineering environment for Qiskit, IBM Quantum Runtime, circuit design, VQE, simulation, visualization, and quantum solution planning.

IBM Bob Quantum Computing (BQCA)

BQCA turns IBM Bob intelligence into a guided quantum-computing workbench. It helps users move from natural-language quantum ideas to structured design artifacts, executable Quantum code, real-time backend implementation, simulation outputs, visualizations, implementation-ready quantum workflows, and automated quantum issues troubleshooting & fixes.

BQCA Solution Appliance IBM Bob Shell Feature Files

BQCA Shell Feature	Shell Description
Appliance name	Bob Quantum Computing Agent (BQCA)
Bob Type	IBM Bob 1.0.6 (Bob Shell)
Root folder in archive	quantum/
Primary launcher	bob-quantum-v1.0.6.sh
Version detector	bob-version.sh
User Interface	xLaunchpad.sh
Authentication helper	bob-auth.sh and bob-auth.md
TLS helper	tls-control.sh
Environment template	.env.example
Bob configuration root	.bob/
QCA tools guide	.bob/quantum-tools/README.md
Main QCA skill	.bob/skills/bob-quantum-skill/SKILL.md
Design directory	bob-quantum-designs/
Implementation directory	bob-quantum-store/
Existing artifacts	Mercury VQE and Uranium-234 VQE design documents, Python code, result files, and visualizations
Integrated quantum products	IBM Qiskit SDK, IBM Qiskit runtime, IBM Quantum Computing Qiskit Connector

BQCA Solution Executive Summary in IBM Bob Lense

What BQCA Delivers to end-Users

BQCA converts IBM Bob from a general AI tool into an enterprise-ready quantum engineering agent with persistent rules, quantum modes, reusable skills, Qiskit-oriented guidance, design-to-code artifact routing, and disciplined output generation.

- Allow users to ask quantum-computing questions and receive domain-specific, structured guidance.
- Generates Qiskit and IBM Quantum Runtime-ready implementation artifacts which are production-ready.
- Designs quantum solution, circuits, algorithm workflows, simulations, and ALL quantum experiment plans.
- Separates architecture/design documents from executable implementation artifacts.
- Includes shared productivity skills and a reusable research workflow contract for planning, skill creation, MCP server creation, Git workflows, and spreadsheet analysis.
- Executes reviewed quantum code on real IBM Quantum hardware through the Qiskit Runtime connector, moving teams beyond simulation to validated QPU results.
- Automatically troubleshoots and repairs quantum code, cutting debugging time and standardizing fixes so quantum work stays repeatable and auditable across a team.

Bottom line: BQCA lets a team move from a plain-language quantum idea to a governed, review-ready result on real IBM Quantum hardware, without every user needing to be a quantum expert or rebuilding the same Qiskit scaffolding from scratch.

Why IBM Bob in BQCA Development?

IBM Bob is the AI execution and reasoning layer for BQCA appliance. Quantum-computing work frequently requires interpretation, decomposition, code generation, circuit design, simulator/real-quantum runtime QPU automatic selection, artifact generation, and clear explanation of quantum as an expert concierge. Bob provides the conversational and agentic layer that lets BQCA translate user intent into governed quantum artifacts.

Why Bob is used in BQCA	Business and technical value
Natural-language quantum interaction	Users can describe desired circuits, quantum experiments, runtime workflows, or quantum learning objectives without manually composing every file.
Reasoning and implementation support	Bob helps decompose quantum objectives into design steps, code artifacts, validation tasks, and outputs.
Custom appliance behavior	QCA-specific modes, rules, skills, and tools guide Bob toward quantum workflows instead of generic software responses.
Artifact discipline	Bob follows QCA routing rules, so design documents and implementation artifacts are stored in the correct project locations.
Enterprise repeatability	Persistent appliance configuration makes quantum workflows more reusable, auditable, and suitable for team enablement.

Custom IBM Bob Functionalities that Produced BQCA

Custom Functionality Name	Description
QCA Custom Modes	Specialized Bob mode definitions that guide user interaction, quantum Q&A, quantum code generation, advanced circuit design, orchestration, planning, and visual/documentation support.
QCA Custom Skills	Reusable procedural skills that instruct Bob how to execute QCA workflows, plan work, create skills/modes, configure MCP, build MCP servers, analyze spreadsheets, and apply Git workflows.
QCA Custom Rules	Mode-specific and appliance-wide rules that enforce quantum behavior, artifact routing, source-grounded responses, design/code separation, no-secret handling, and consistent output expectations.
QCA Custom Tool Guidance	Quantum-focused tool under .bob/quantum-tools that steers Bob toward Qiskit SDK, IBM Quantum Runtime, real quantum backend, simulator workflows, visualization, and reproducible artifact generation and productive quantum code execution.

BQCA Capabilities in Quantum Technology by IBM Bob

Quantum Capability	What QCA Enables	Executive Value
Quantum Q&A	Explains circuits, gates, qubits, measurement, noise, simulators, algorithms, and IBM Quantum concepts with Bob intelligence.	Accelerates onboarding and quantum literacy.
Quantum Code Generation	Create Python/Qiskit code for circuits, algorithms, simulation, optimization, visualization, and runtime-style workflows.	Reduces prototype time and standardizes quantum development.
Quantum Circuit Design	Helps design, reason about, and document circuits for learning, research, and applied experiments.	Improves repeatability and communication of quantum designs.
VQE and Chemistry Workflows	Supports molecular simulation pattern designs intelligently guided by Bob	Connects quantum experimentation to material science, chemistry, and energy exploration.
Quantum Optimization	Frames optimization problems, QAOA-style workflows, and hybrid classical/quantum solution paths.	Supports logistics, finance, scheduling, and resource allocation innovation.
Simulation and Visualization	Produces outputs, plots, result files, and PNG visualizations for review and reporting guided by Bob.	Makes quantum outcomes understandable to technical and non-technical stakeholders.
Quantum Backend real-time Execution	Execute the quantum code on a real QPU backend with quantum computing Qiskit Connector and Bob intelligence	Delivers real quantum experience to the end-users to enhance productivity
Design-to-Code Routing	Stores architecture/design separately from executable quantum code.	Improves governance, reviewability, and artifact reuse.

BQCA Quantum Intelligence by IBM Bob

BQCA quantum intelligence is the appliance-specific behavior that makes Bob act like a quantum-aware engineering partner. It combines quantum terminology, Qiskit patterns, simulation workflows, IBM Quantum Runtime awareness, algorithm reasoning, circuit construction, artifact discipline, and reusable project context. For users, this means BQCA can help clarify a quantum objective, recommend an implementation path, generate code, create design documentation, and produce review-ready artifacts without losing the structure required for enterprise delivery.

User Outcome

The user does not need to start from a blank editor or be an expert in quantum to use the QCA appliance. BQCA provides a guided path from quantum idea to circuit design, from design to quantum implementation, and from implementation to validated result artifacts ready for quantum production powered by IBM Bob.

BQCA Use Cases in Industries & Business Values

Use Case	Industry	Business Value
Molecular algorithm exploration	Chemicals, Energy, Materials Science, Pharmaceuticals	Supports discovery research, candidate screening, and future quantum advantage exploration.
Portfolio and risk optimization	Financial Services, Banks, Insurance, Stockbrokers	Explores hybrid quantum optimization approaches for portfolio construction and risk-sensitive decision support.
Supply-chain optimization	Manufacturing, Retail, Logistics	Models complex routing, scheduling, and resource-allocation problems for innovation pilots.
Quantum education and workforce enablement	Enterprise IT, Universities, Research Labs	Accelerates workforce readiness through guided Qiskit examples and quantum explanations.
Secure innovation planning	Government, Defense, Regulated Industries	Supports controlled exploration of quantum concepts, algorithms, and future computing roadmaps.
AI and quantum research prototyping	AI Labs, Research Organizations	Enables rapid experiment planning, code generation, simulation, and visualization.
Energy materials analysis	Energy, Utilities, Advanced Materials	Supports exploratory quantum chemistry workflows for advanced material and catalyst research.

BQCA Quantum Research Productivity Problems Solved

Productivity Problem	How BQCA Solves It	Executive Impact
Slow quantum onboarding	Using automation provides guided explanations, code examples, and structured workflow paths.	Reduces time-to-competency for teams new to quantum.
Fragmented quantum design and code	Using automation, routes design documents and implementation files into separate governed directories.	Improves review, reuse, and lifecycle management.
Manual Qiskit boilerplate	Using automation, generates Qiskit scaffolding, circuit code, simulator patterns, and visualization steps.	Accelerates prototyping and reduces repetitive coding.
Inconsistent experiment documentation and quantum code troubleshooting difficulties	Using automation, produces structured plans, designs, outputs, and result artifacts as well as intelligently troubleshoots quantum codes	Improves auditability and communication with stakeholders.
Unclear quantum use-case framing	Using automation, help users map industry problems to circuits, VQE, QAOA, simulation, or hybrid workflows.	Improves business-value alignment for quantum pilots.
Knowledge scattered across tools	Uses custom BQCA tools, modes, rules, and skills to centralize quantum workflow guidance.	Creates a consistent appliance experience for teams.
Quantum code workload automated troubleshooting & fixes	Uses custom BQCA tools, modes, rules, and skills to automatically troubleshoot & fix quantum workloads.	Produces production-level quantum code and circuit designs.

BQCA Other Research Business Use and Productivity Values

Productivity	Business Relevance
Design Governance	BQCA separates architecture/design assets from executable implementation artifacts so teams can review business intent before running code.
Reusable Quantum Patterns	Grover and VQE examples show that other IBM Research teams can reuse the same BQCA modes, validators, notebook automation, backend policy, and artifact contract while substituting a new algorithm, molecule, Hamiltonian, or optimization problem.
Executive Communication	BQCA outputs can be converted into README files, Word documents, diagrams, and visualization assets for leadership updates.
Developer Enablement	BQCA supports Qiskit-oriented coding, Git discipline, and reusable skills that improve developer consistency.
Security and Secrets	Environment variables are managed through .env and .env.example patterns; real credentials should not be committed or disclosed.
Documentation References	Relevant documentation includes IBM Bob configuration, custom modes, rules, skills, MCP, Qiskit, IBM Quantum, and IBM Quantum Runtime resources.

How to Use BQCA Appliance

Usage Objective

The BQCA appliance is intended to be cloned from the repository, configured with approved environment variables, launched through xLaunchpad, and used through menu-driven Bob Shell interactions. The same configuration template, automation tools, and governed output contract allow other IBM Research teams to reuse the appliance without rebuilding the workflow.

Basic Setup

Step	Action	Command
1	Clone the repository	git clone <repository-url>
2	Enter the QCA directory	cd quantum
3	Create the environment file	cp .env.example .env
4	Edit required values	vi .env # or use your preferred editor
5	Make launch scripts executable	chmod +x xLaunchpad.sh bob-quantum-v*.sh bob-auth.sh tls-control.sh bob-version.sh
6	Launch QCA	./xLaunchpad.sh

Required Configuration Checklist

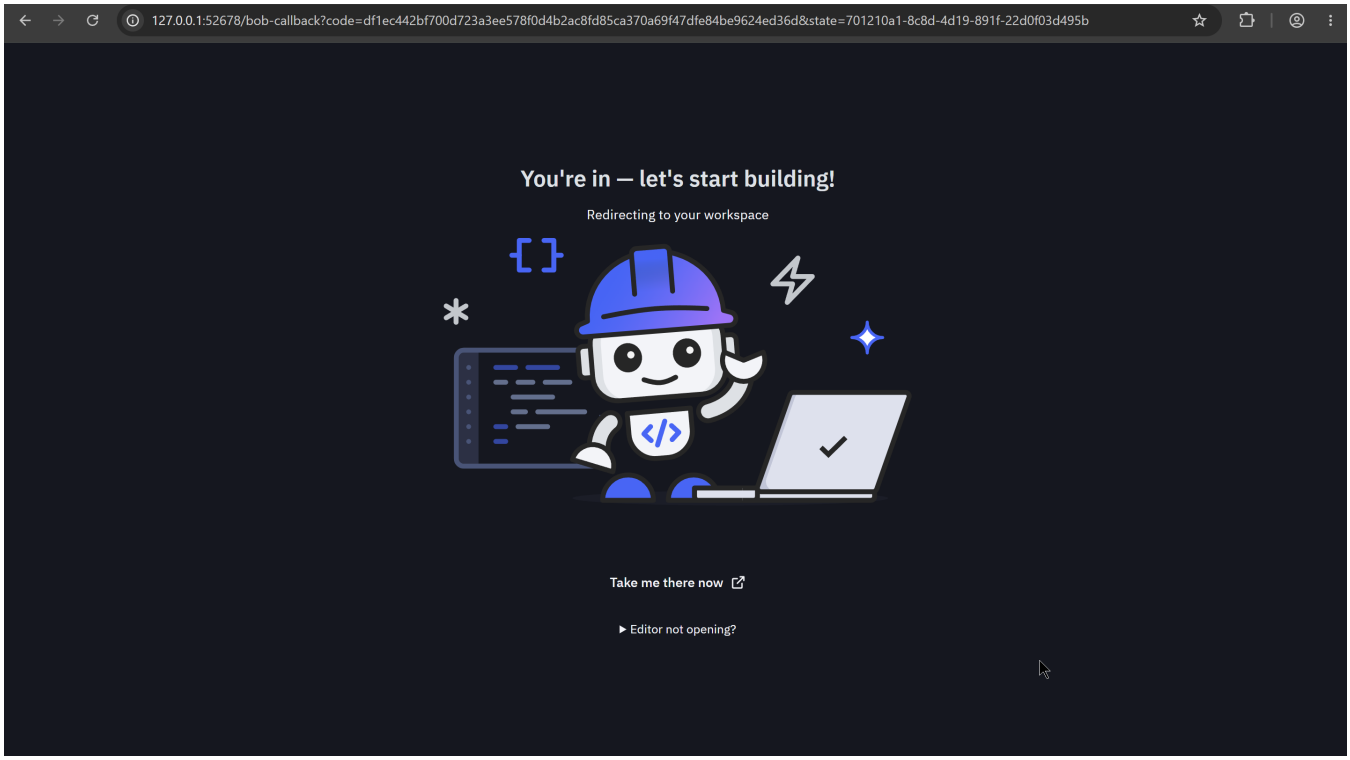
Configuration Item	Purpose
BOB_API_KEY	Authenticates Bob Shell access.
BOB_INSTANCE_ID	Identifies the QCA appliance runtime session.
BOB_BIN	Optional override for Bob executable path. Defaults to bob.
BOB_INCLUDED_DIRECTORY	Defines QCA design/store directories included in Bob context.
QCA_DESIGN_DIR	Directory for design and architecture outputs.
QCA_STORE_DIR	Directory for code, notebooks, scripts, results, and generated artifacts.
TLS settings	Used only when enterprise TLS configuration is required.

Recommended Operating Practices, Security, & Guardrails

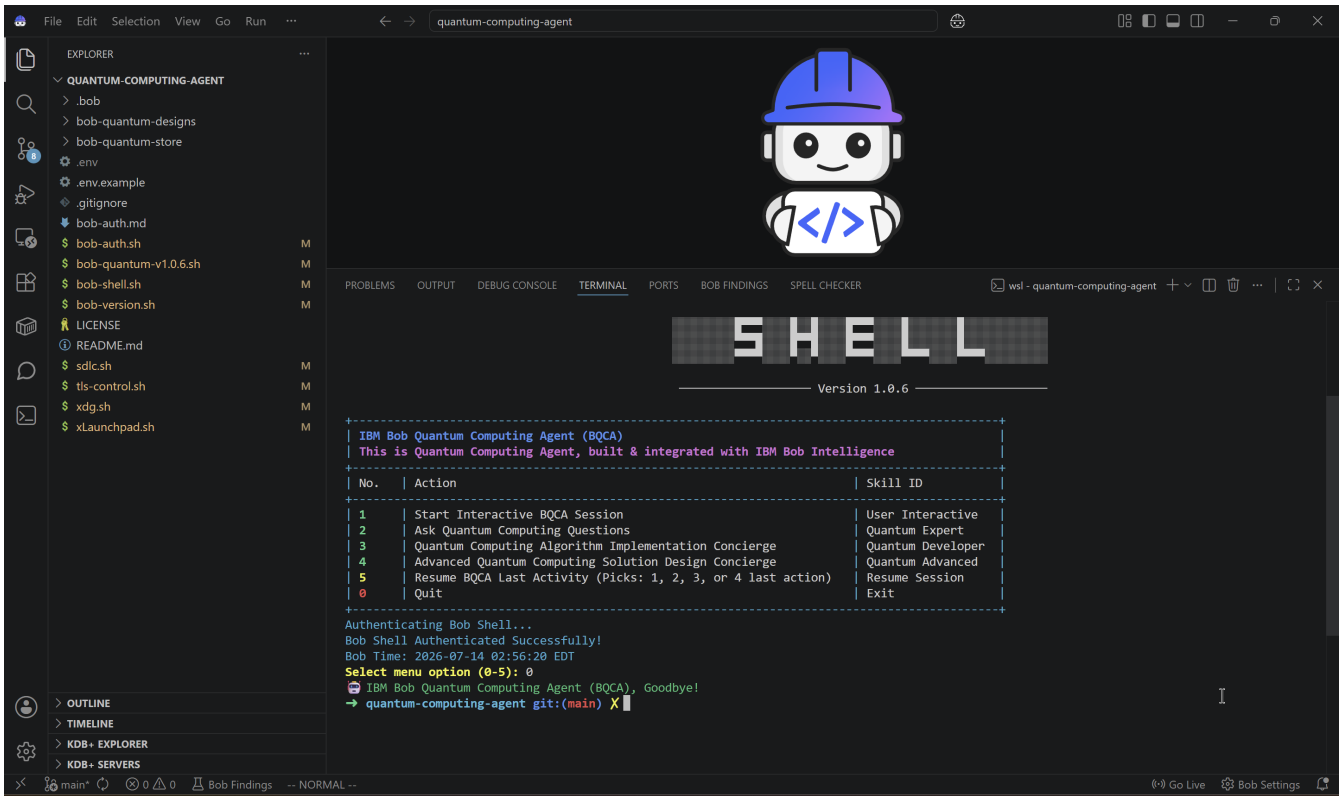
- Use **#4** when the request is quantum architectural, conceptual, or design planning oriented.
- Use **#3** when requesting executable Quantum code, or real Quantum backend workload execution.
- **Security**: Keep secrets only in .env or secure enterprise secret stores; never commit real credentials.
- Use **#2** when you are a quantum starter and want to ask BQCA about your real-world quantum problems.
- Use **#1** when you want to ask BQCA anything quantum. BQCA will help turn your problem into solution design.

ADDENDUM

➤ Bob Authentication Redirect to BQCA Workspace

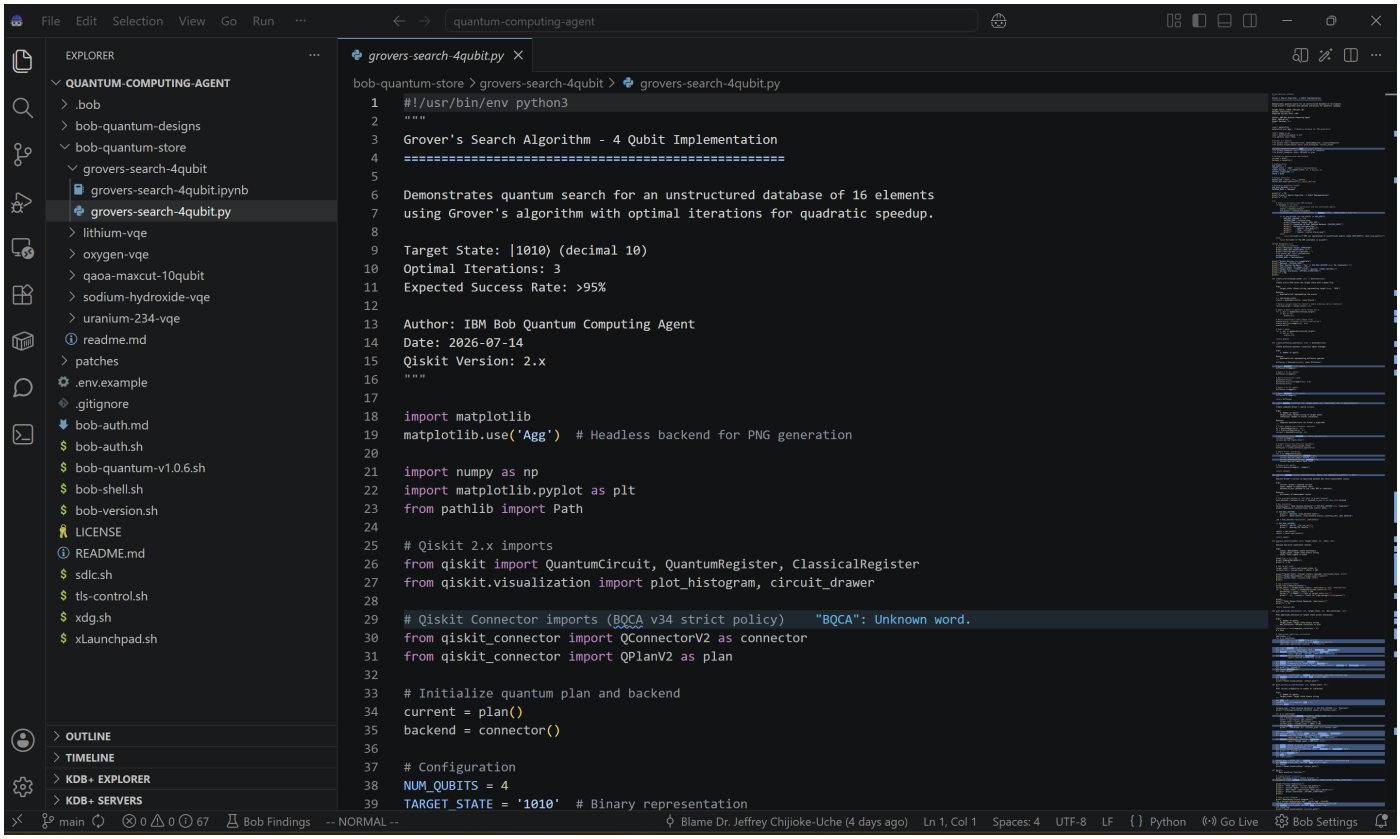


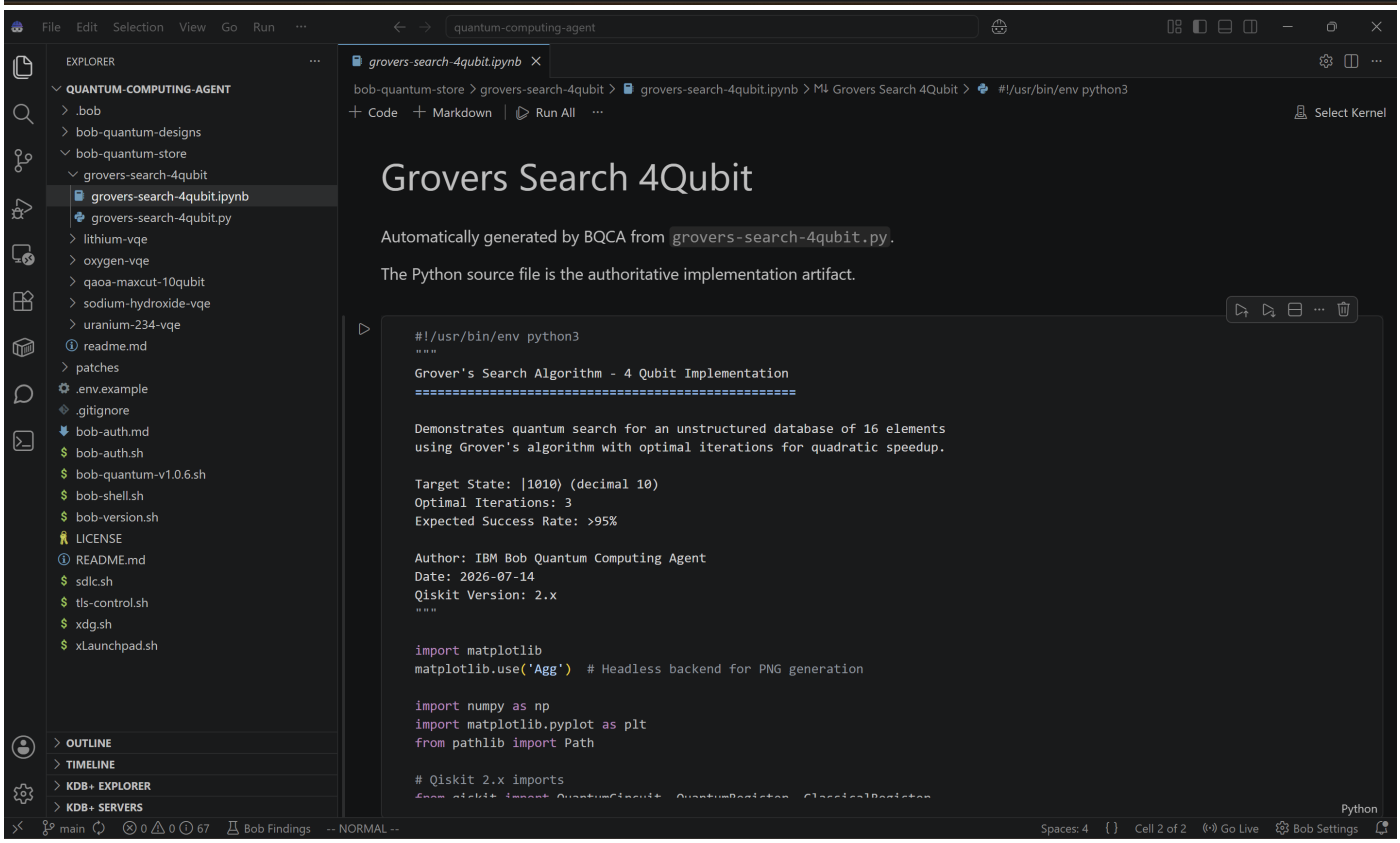
➤ BQCA Solution Code Structure in Workspace after Authentication



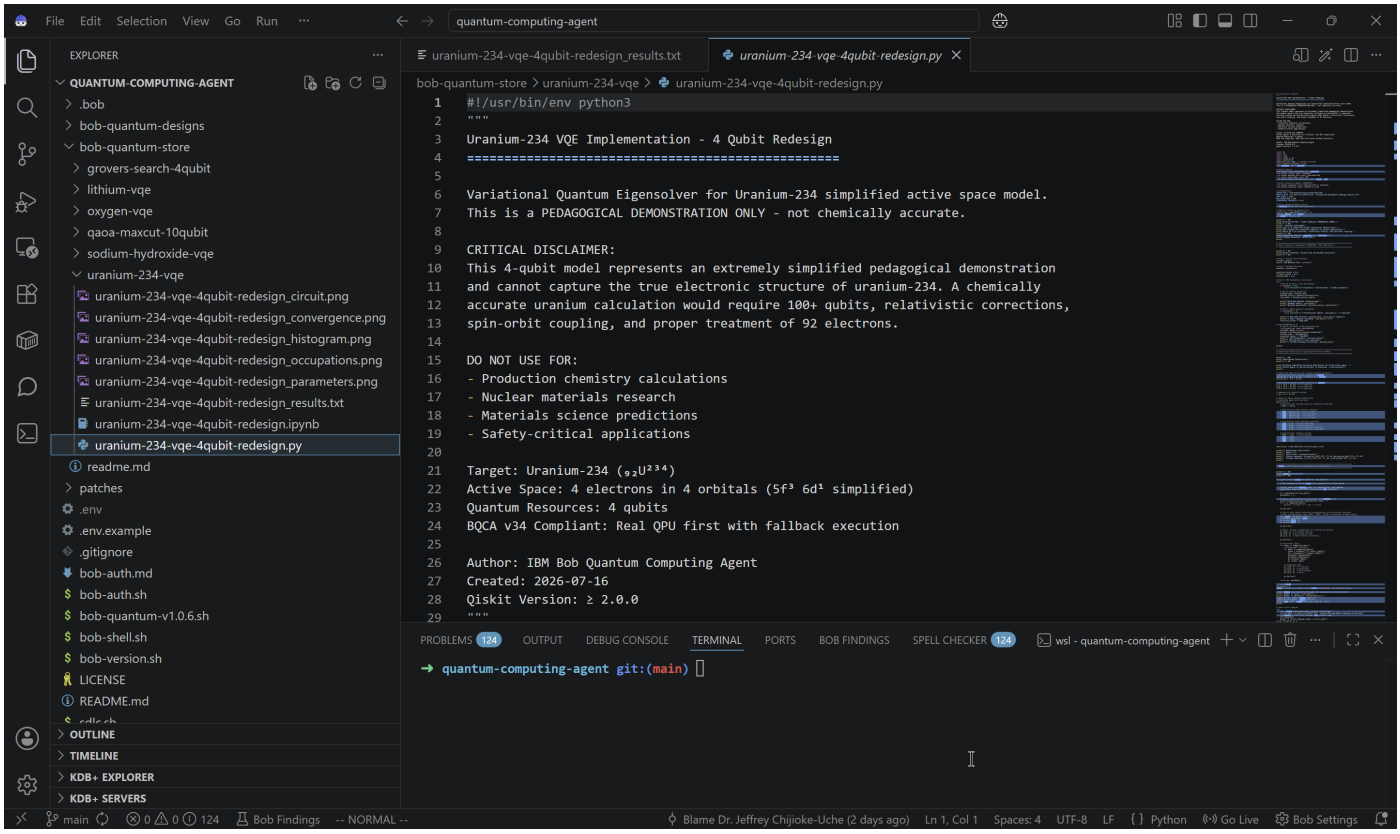
➢ Automation Evidence (List a Few)

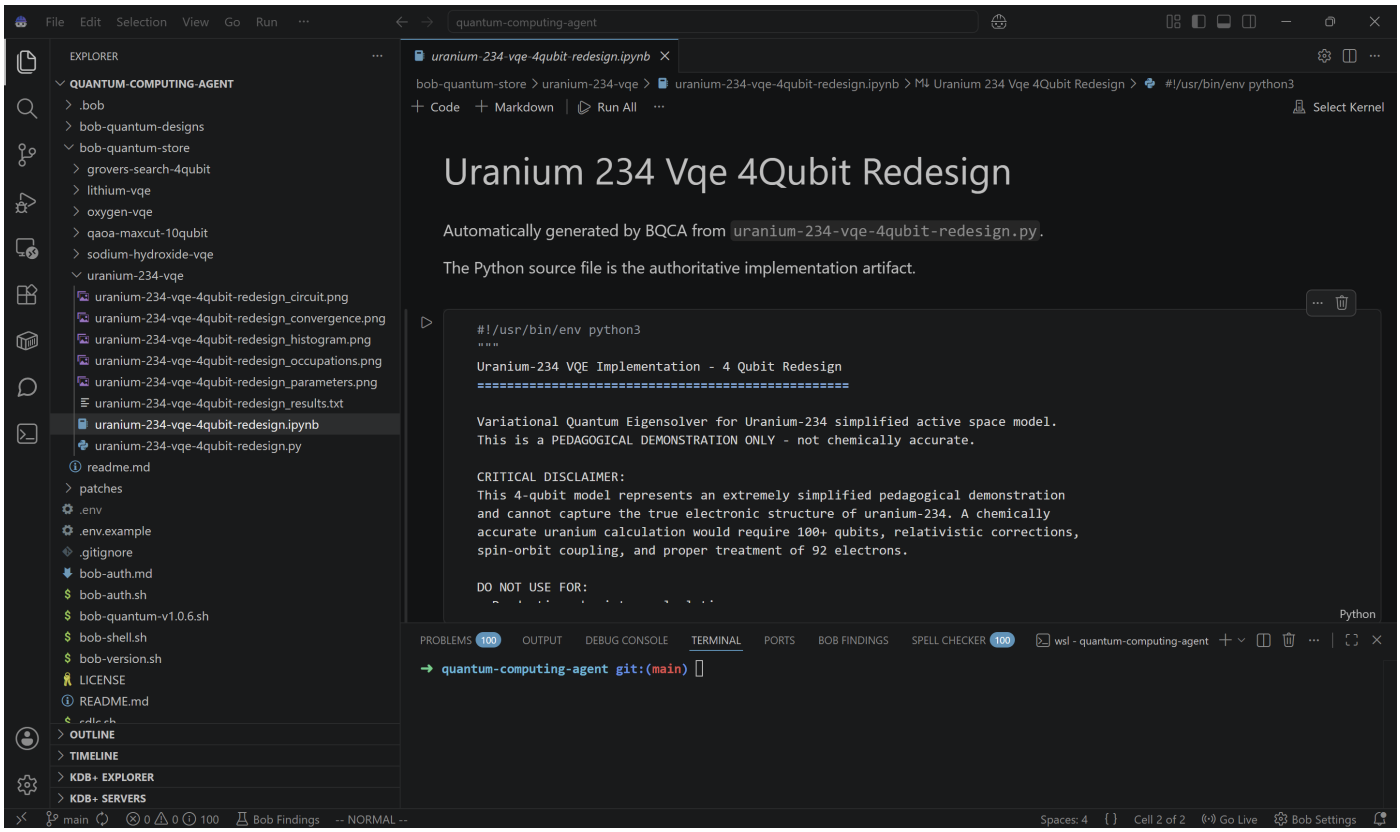
➢ *3. Grover's Search Algorithm: Automatic Generated Quantum Jupyter Notebook Code*



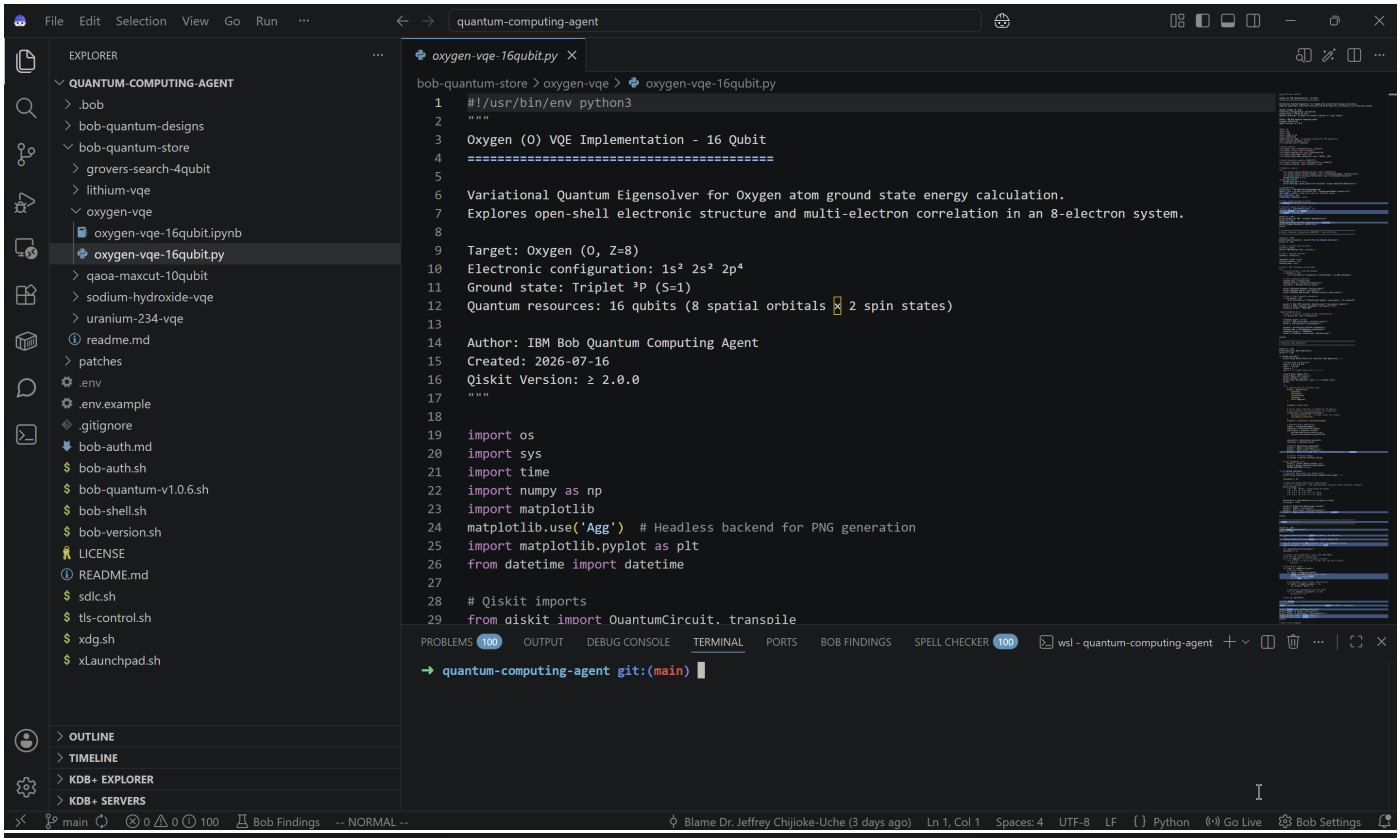


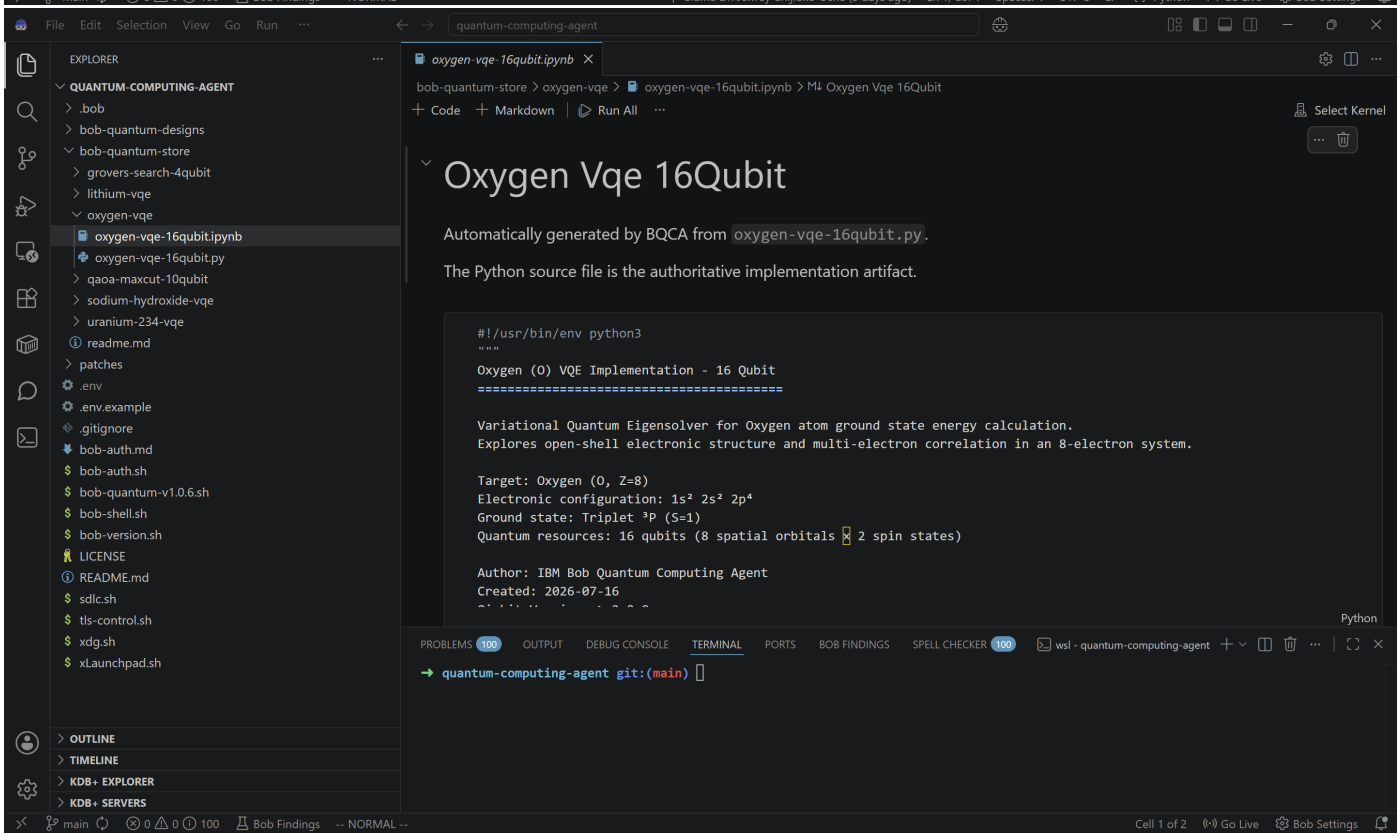
➢ *Uranium 234 VQE: Automatic Generated Quantum Jupyter Notebook Code*



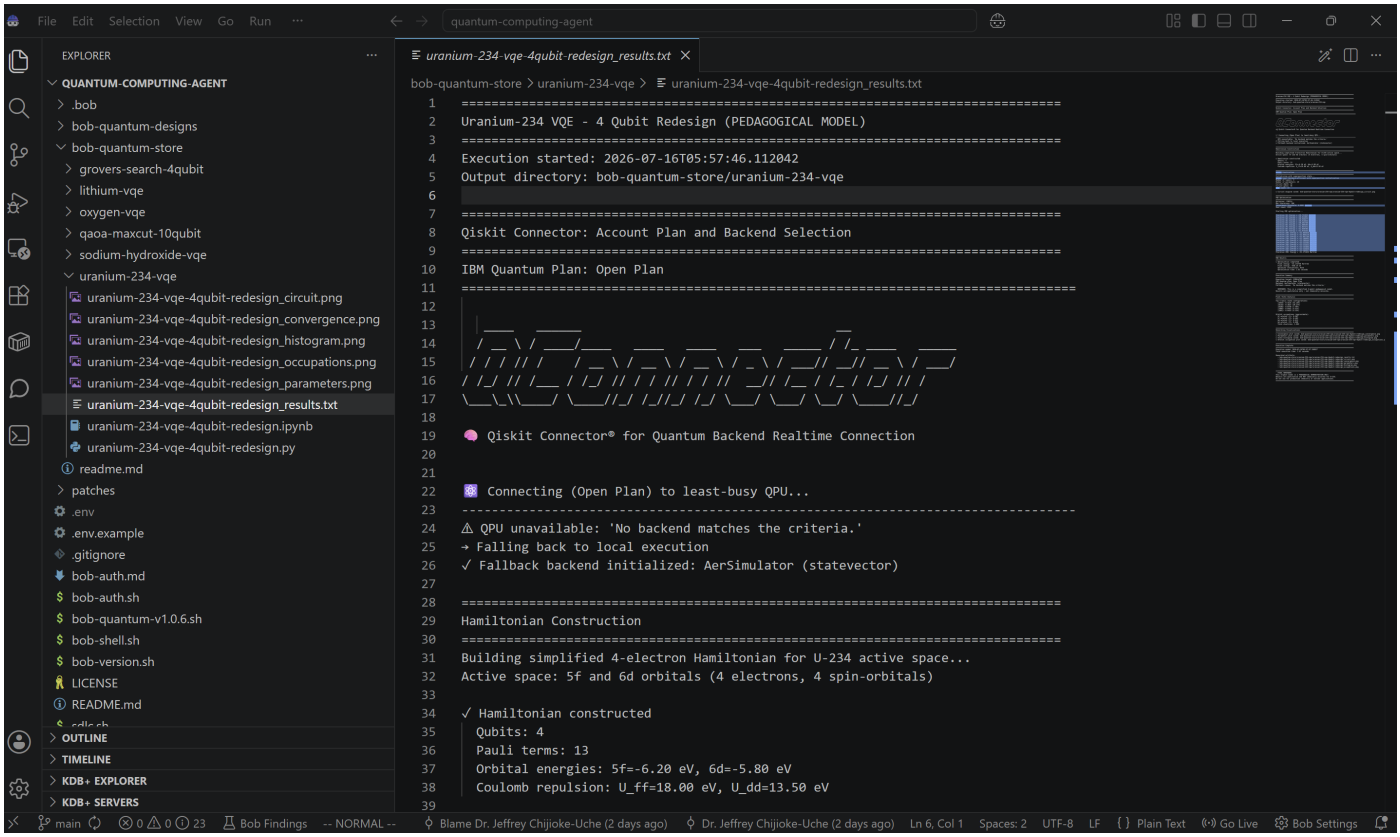


➢ *Oxygen VQE: Automatic Generated Quantum Jupyter Notebook Code*

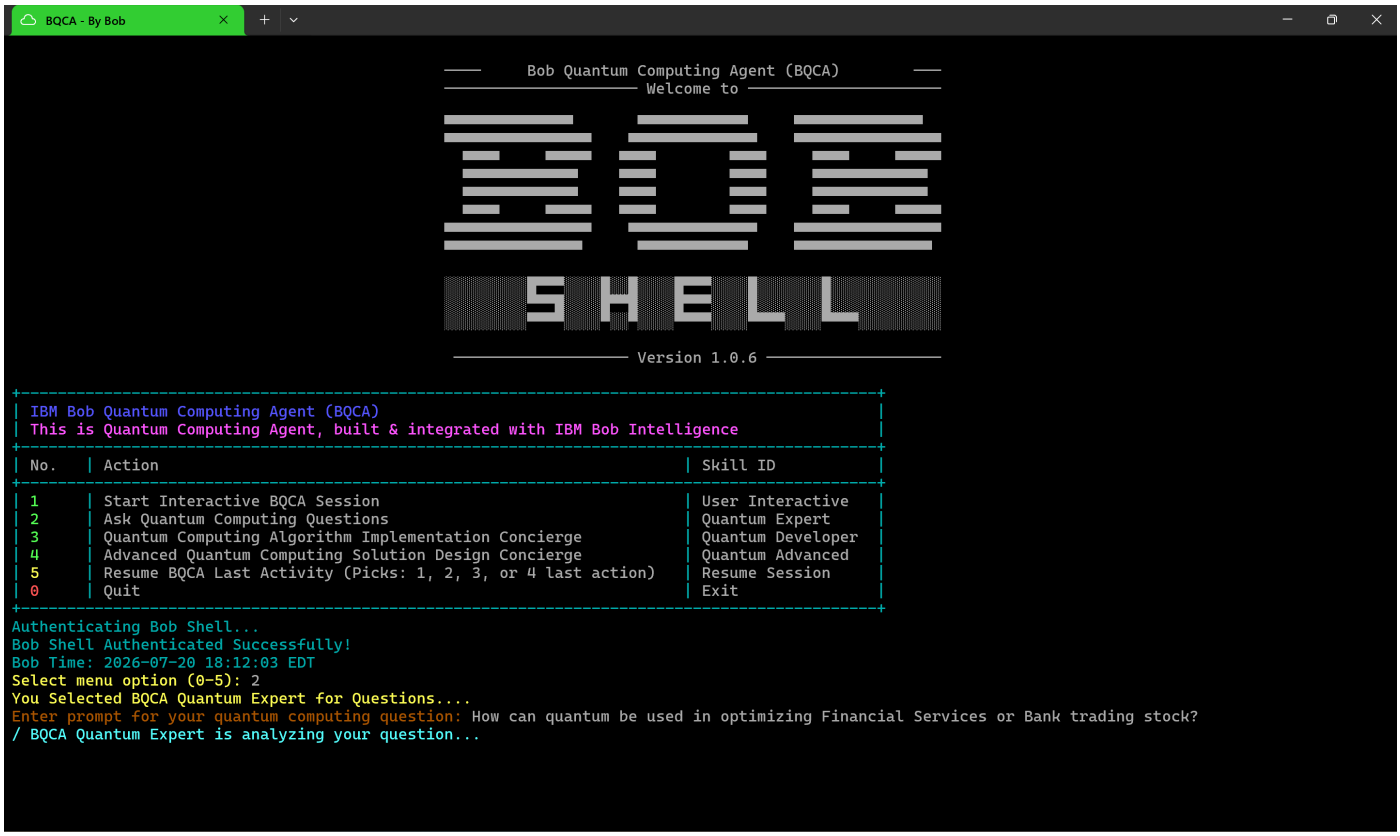




➢ *Real Quantum QPU Connectivity: Automatic Connectivity*



➢ BQCA Processing User Prompt



➢ BQCA Architecture Analysis – Proof of a Bob-centric Solution

Architecture Layer Map

Layer	Component	Technology / IBM Product	Role in BQCA
1 — User Actor	Human researcher / developer	Terminal + xLaunchpad.sh	Submits natural-language Bob prompt via the BQCA numbered menu.
2 — BQCA Launchpad	xLaunchpad.sh + bob-quantum-v1.0.6.sh	IBM Bob Shell v1.0.6	Loads .env credentials, detects Bob version, presents interactive menu, invokes Bob with the correct mode and prompt.
3 — IBM Bob Engine	IBM Bob reasoning and orchestration layer	IBM Bob (AI agent)	Classifies intent, clarifies missing parameters, decomposes multi-stage quantum work, invokes BQCA skills, enforces rules, generates artifacts.
4 — BQCA Modes	Custom Bob mode definitions (.bob/custom_modes.yaml)	IBM Bob custom modes	Routes Q&A to Quantum Sky mode, code requests to Quantum Code/Heron mode, design requests to Quantum Advanced mode.
5 — BQCA Skills	bob-quantum-skill + shared skills (.bob/skills/)	IBM Bob custom skills	Provides step-by-step workflow instructions to Bob: Qiskit code generation, VQE/QAOA design, notebook generation, MCP, Git, planning.
6 — BQCA Rules	Mode-scoped rule files (.bob/rules-*/)	IBM Bob custom rules	Enforces no-secret handling, design/code routing, real-QPU-first policy, artifact verification, and simulator fallback requirements.
7 — Qiskit Code Gen	Bob generates authoritative .py Qiskit source	Qiskit SDK 2.x	Produces validated Python: circuit construction, algorithm logic, execution wiring, result handling, error handling, and documentation.
8 — Validation & Repair	Syntax + policy validation loop	BQCA internal validator	Checks Python validity, required Connector behavior, QPU-first logic, fallback presence, notebook currency. Repairs and reruns on failure.
9 — Notebook Gen	Auto-generated .ipynb sibling	Jupyter / nbformat	Creates same-name notebook from .py source. Validated and kept synchronized; regenerated if source changes.
10 — Qiskit Connector	Account + backend resolution	IBM Quantum Computing Qiskit Connector™	Resolves IBM Quantum account plan (Open/Flex/Premium), selects accessible backend, initializes Runtime session credentials.
11 — Qiskit Runtime VE	Virtual Python environment	Qiskit Runtime + venv	Provisioned automatically: venv creation, pip upgrades, Qiskit SDK install, Runtime install, Connector install, import verification.

12 — QPU Execution	Real IBM Quantum hardware attempt	IBM Quantum QPU (Heron / Eagle)	Checks QPU availability, operational status, and qubit capacity. Attempts real execution first. Records REAL_QPU in output.
13 — Simulator Fallback	Controlled try/except fallback path	Qiskit Aer / StatevectorSimulator	Executes on simulator when real QPU is unavailable. Records SIMULATOR + fallback reason. Workflow never stops due to hardware status.
14 — Artifact Routing	Governed output directories	bob-quantum-designs/ + bob-quantum-store/	Design docs route to bob-quantum-designs/. Python, notebooks, results, plots, logs route to bob-quantum-store/. Naming is predictable and auditable.
15 — Session Resume	Menu option 5	IBM Bob session context	Restores the most recent activity (menu 1/2/3/4) so the user resumes without reconstructing context, re-entering parameters, or relocating files.

➢ BQCA Menu Options: Bob Mode and Prompt-to-Output Mapping

Each BQCA menu option activates a distinct IBM Bob mode with a specific skill, rule set, and expected output contract. The table below shows how a user's menu selection maps to the full Bob + Qiskit execution path.

Menu #	Action	Bob Mode Activated	Bob Skill Invoked	Rules Enforced	Output Produced
1	Start Interactive BQCA Session	IBM Bob Quantum Sky (ask)	bob-quantum-skill	rules-ask, rules-quantum-intelligence	Open-ended Q&A, concept explanations, circuit walkthroughs, algorithm comparisons, research guidance.
2	Ask Quantum Computing Questions	IBM Bob Quantum Sky (ask)	bob-quantum-skill	rules-ask, rules-qiskit-connector	Domain-specific answers, Qiskit API guidance, algorithm selection, IBM Quantum backend advice, design concepts.
3	Quantum Algorithm Implementation Concierge	IBM Bob Quantum Code / Code Heron (code)	bob-quantum-skill	rules-code, rules-quantum-code, rules-qiskit-connector	Validated .py source, auto-generated .ipynb notebook, Connector-initialized backend, QPU/simulator result, plots, execution notes.
4	Advanced Quantum Solution Design Concierge	IBM Bob Quantum Advanced (advanced)	bob-quantum-skill	rules-advanced, rules-quantum-intelligence	Structured design document in bob-quantum-designs/: problem statement, algorithm, qubit assumptions, circuit plan, backend strategy, validation criteria, risks.
5	Resume BQCA Last Activity	Restores last used mode	Restores last used skill	All rules from last mode	Resumes prior session context without reconstructing parameters, re-entering problem statements, or relocating artifacts.

IBM Bob Prompt Examples for Quantum Code Generation (Menu Option 3)

These are real examples of prompts submitted to IBM Bob via the BQCA menu that produced validated Qiskit Python artifacts and auto-generated Jupyter notebooks. Each prompt triggers the full 10-stage pipeline.

Prompt Example 1 — Grover Search Algorithm

Bob Prompt: "*Using the BQCA Quantum Developer concierge, create a Grover Search algorithm for a 3-qubit oracle that marks the state |101>. Generate the Qiskit v2 implementation, run it on a real IBM Quantum backend using Qiskit Connector, fall back to simulator if QPU is unavailable, and save the result and circuit diagram*."

Bob Response Pipeline: *Bob clarifies qubit count and oracle target -> activates Quantum Code mode -> invokes bob-quantum-skill -> generates grover_search.py with Qiskit SDK 2.x circuit, oracle, amplification, and measurement -> validates Python and QPU-first policy -> auto-generates grover_search.ipynb -> Qiskit Connector resolves backend -> attempts real QPU -> falls back to Aer simulator -> saves results to bob-quantum-store/grover-results.txt and circuit PNG.*

Artifact: bob-quantum-store/grover_search.py (validated Qiskit Python source)

Artifact: bob-quantum-store/grover_search.ipynb (auto-generated Jupyter notebook)

Artifact: bob-quantum-store/grover-circuit.png (circuit visualization)

Execution output: REAL_QPU or SIMULATOR + fallback reason recorded

Prompt Example 2 — Uranium-234 VQE Molecular Simulation

Bob Prompt: "*Design and implement a VQE workflow for Uranium-234 nuclear binding energy estimation. Use the UCCSD ansatz, map to qubits via Jordan-Wigner, configure Qiskit Connector for my IBM Quantum account, attempt real QPU first and fall back to StatevectorSimulator, and generate both the .py source and .ipynb notebook with a convergence plot*."

Bob Response Pipeline: *Bob confirms molecule (U-234), ansatz (UCCSD), qubit mapping (Jordan-Wigner), and backend preference -> activates Quantum Code Heron mode -> invokes bob-quantum-skill -> generates uranium_234_vqe.py with Hamiltonian construction, VQE optimizer loop, Connector initialization, QPU-first execution, and matplotlib convergence plot -> validates and repairs -> auto-generates uranium_234_vqe.ipynb -> routes artifacts to bob-quantum-store/.*

Artifact: bob-quantum-store/uranium_234_vqe.py (validated VQE implementation)

Artifact: bob-quantum-store/uranium_234_vqe.ipynb (interactive notebook)

Artifact: bob-quantum-store/uranium_234_vqe-convergence.png (energy convergence plot)

Execution output: ground-state energy estimate + REAL_QPU / SIMULATOR status

Prompt Example 3 — Oxygen VQE Chemistry Simulation

Bob Prompt: "*Create a VQE implementation for Oxygen (O2) ground-state energy using a hardware-efficient ansatz and Qiskit Runtime. Use my IBM Quantum Open Plan account via Qiskit Connector, save the Python source, notebook, and result file*."

Bob Response Pipeline: *Bob confirms molecule (O2), ansatz (hardware-efficient), plan (Open), and runtime preference -> generates oxygen_vqe.py -> validates against all BQCA rules including no-secret, QPU-first, and fallback -> auto-generates oxygen_vqe.ipynb -> executes and records SIMULATOR (Open Plan hardware not sufficient for this circuit) with fallback reason -> packages artifacts.*

Artifact: bob-quantum-store/oxygen_vqe.py

Artifact: bob-quantum-store/oxygen_vqe.ipynb

Execution output: SIMULATOR (fallback: circuit exceeds Open Plan qubit budget) — transparently recorded

Prompt Example 4 — QAOA MaxCut Optimization

Bob Prompt: "*Using BQCA, generate a QAOA circuit for a MaxCut problem on a 4-node weighted graph. Use p=2 layers, COBYLA optimizer, and Qiskit SDK 2.x patterns. Attempt real QPU execution via Qiskit Connector and fall back to AerSimulator. Save the Python, notebook, and the probability distribution histogram.*"

Bob Response Pipeline: *Bob confirms graph (4-node), layers (p=2), optimizer (COBYLA), and backend preference -> generates qaoa_maxcut.py with graph construction, QAOA circuit, parameter optimization loop, and histogram plot -> validates and auto-generates qaoa_maxcut.ipynb -> executes and saves probability histogram PNG to bob-quantum-store/.*

Artifact: bob-quantum-store/qaoa_maxcut.py

Artifact: bob-quantum-store/qaoa_maxcut.ipynb

Artifact: bob-quantum-store/qaoa_maxcut-histogram.png

Prompt Example 5 — GHZ Circuit Design and Real QPU Execution

Bob Prompt: "*Create a 5-qubit GHZ state circuit using Qiskit v2. Draw the circuit, measure all qubits, run on IBM Quantum backend via Qiskit Connector with 1024 shots, fall back to AerSimulator if needed, and save the circuit PNG and measurement histogram."*

Bob Response Pipeline: *Bob confirms qubit count (5) and shot count (1024) -> generates ghz_circuit.py -> validates -> generates ghz_circuit.ipynb -> Connector resolects backend -> submits job -> records REAL_QPU if successful, SIMULATOR with reason if not -> saves circuit PNG and histogram to bob-quantum-store/.*

Artifact: bob-quantum-store/ghz_circuit.py

Artifact: bob-quantum-store/ghz_circuit.ipynb

Artifact: bob-quantum-store/ghz_circuit.png + ghz_circuit-histogram.png

IBM Bob Prompt Examples for Quantum Design (Menu Option 4)

These prompts activate the Advanced Quantum Solution Design Concierge mode and produce governed Markdown design documents in bob-quantum-designs/ — no code is executed until the researcher reviews and approves the design.

Design Prompt 1: "*Design a quantum circuit architecture for estimating the ground-state energy of lithium hydride (LiH) using VQE with UCCSD ansatz. Include qubit count, circuit depth estimate, backend strategy, simulator fallback plan, validation criteria, and expected artifacts."*

Bob Output: *Structured Markdown design document saved to bob-quantum-designs/lih_vqe_design.md containing: problem statement, algorithm selection rationale, qubit and gate assumptions, QPU vs simulator decision tree, risk assessment, and artifact checklist.*

Design Prompt 2: "*Create a design document for a QAOA-based supply chain routing optimization problem with 6 nodes. Include QUBO formulation, ansatz depth, optimizer choice, IBM Quantum backend recommendation, and expected quantum advantage threshold.*"

Bob Output: *bob-quantum-designs/qaoa_supply_chain_design.md with QUBO mapping, circuit depth analysis, optimizer comparison (COBYLA vs SPSA), IBM Quantum backend selection guidance, and classical baseline comparison notes.*

How IBM Bob Drives the BQCA Quantum Workflow

IBM Bob is not a passive code template in BQCA. It is the active orchestration engine that makes every stage of the quantum workflow intelligent, adaptive, and governed. The table below shows exactly where Bob adds value at each pipeline stage.

Pipeline Stage	What IBM Bob Does	Why It Matters to IBM Research

Intent classification	Reads the user's natural-language prompt and determines whether it is a question (menu 2), a code request (menu 3), or a design request (menu 4).	No routing configuration required. A researcher describes their quantum goal in plain language and Bob handles the rest.
Parameter clarification	Asks for missing details (molecule, qubit count, backend, shots, optimizer) before generating anything.	Eliminates the most common failure mode: long generation cycles that terminate because an input was never specified.
Algorithm selection	Reasons about the problem type and recommends the appropriate quantum algorithm (VQE, QAOA, Grover, QPE, HHL) with justification.	A non-expert researcher receives an expert-level algorithm recommendation grounded in the problem structure.
Qiskit code generation	Writes the complete, production-quality .py source using current Qiskit SDK 2.x patterns, with Connector initialization, QPU-first logic, fallback, result handling, and documentation.	The researcher starts from a validated, runnable implementation rather than a blank editor. Boilerplate and version-reconciliation time drops to zero.
Validation and repair	Checks the generated code against syntax and policy rules. If anything fails, Bob reads the failure, repairs the implementation, and revalidates — without user intervention.	The automated repair loop compresses the manual edit-execute-debug cycle from hours to seconds.
Notebook generation	Immediately after .py validation, Bob triggers the notebook generator to create the same-name .ipynb, keeping source and notebook synchronized.	The researcher receives two delivery formats from one request: a production Python script and an interactive notebook for explanation and demonstration.
Artifact routing	Routes design documents to bob-quantum-designs/ and all implementation artifacts to bob-quantum-store/ with predictable naming.	Any team member can locate, review, re-run, or hand off any prior BQCA workload without asking where files are stored.
Cross-team enablement	The same Bob configuration, modes, skills, rules, and artifact contract are reused by any IBM Research team that clones the repository.	Zero custom development is required for a new team to start generating governed quantum artifacts from day one.

BQCA Architecture in one sentence: IBM Bob receives a natural-language quantum prompt from the user via the BQCA menu, decomposes and orchestrates the full research-to-artifact pipeline using custom modes, skills, and rules, generates and validates Qiskit Python code, auto-generates the Jupyter notebook, resolves the IBM Quantum account and backend through Qiskit Connector, attempts real QPU execution first and falls back to simulator under a controlled policy, and routes all artifacts — design, code, notebook, results, plots — to governed directories for human review, Git commit, and cross-team reuse.

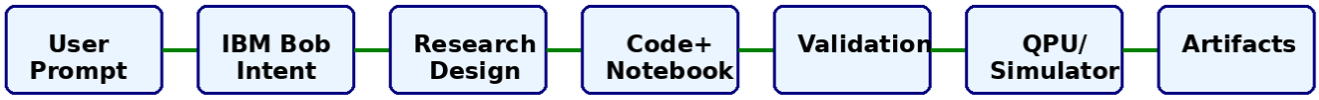
All five BQCA menu options, all ten pipeline stages, all artifact types, and all IBM technologies (IBM Bob, Qiskit SDK, IBM Quantum Computing Qiskit Connector, Qiskit Runtime, IBM Quantum QPU, Qiskit Aer Simulator) are represented in the architecture above. This architecture directly addresses the IBM Research judging committee feedback requesting explicit Bob-driven prompts, code generation, and an end-to-end BQCA menu flow.

References

- https://pypi.org/project/qiskit-connector/
- https://bob.ibm.com/docs/ide/faq
- bob-quantum-team/quantum-computing-agent
- IBM Quantum Computing | Home
- https://qiskit-community.github.io/qiskit-algorithms/stubs/qiskit_algorithms.AmplitudeEstimation.html

ADDITIONAL JUDGE EVIDENCE

➢ Figure A1. BQCA Research Automation Workflow



➢ Table A1. BQCA Workflow Evidence

Workflow Stage	Manual	BQCA	Improvement
Research framing	2-4 hrs	10 min	95%
Algorithm selection	3 hrs	8 min	96%
Solution design	4 hrs	12 min	95%
Code generation	6 hrs	15 min	96%
Validation	5 hrs	20 min	93%
Documentation	3 hrs	10 min	94%
Notebook	2 hrs	<5 min	96%
Packaging	2 hrs	5 min	96%

➢ Table A2. BQCA Cross-Team Reuse

Artifact	**Automatically Generated**	**Reusable**
Design	Yes	Yes
Python	Yes	Yes
Notebook	Yes	Yes
Plots	Yes	Yes
Logs	Yes	Yes
Documentation	Yes	Yes